

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
data = pd.read_excel("Banking Data.xlsx")
print(data)
```

	Client ID	Name	Age	Sex	Location ID	Joined Bank \
0	PKR81288	Raymond Mills	24	Male	34324	2019-05-06
1	PKR65833	Julia Spencer	23	Male	42205	2001-12-10
2	PKR47499	Stephen Murray	27	Female	7314	2010-01-25
3	PKR72498	Virginia Garza	40	Male	34594	1993-03-28
4	PKR60181	Melissa Sanders	46	Female	41269	2012-07-20
...	...	...	...	...	...	...
2995	PKR66827	Earl Hall	82	Female	8760	2014-10-09
2996	PKR40556	Billy Williamson	44	Female	32837	2009-02-05
2997	PKR72414	Victor Black	70	Female	36088	2009-12-29
2998	PKR46652	Andrew Ford	56	Female	24871	2006-02-13
2999	PKR40216	Amy Nguyen	79	Female	38518	2005-12-08

	Banking Contact	Nationality	Occupation \
0	Anthony Torres	American	Safety Technician IV
1	Jonathan Hawkins	African	Software Consultant
2	Anthony Berry	European	Help Desk Operator
3	Steve Diaz	American	Geologist II
4	Shawn Long	American	Assistant Professor
...	...	...	...
2995	Joshua Bennett	American	Accounting Assistant III
2996	Dennis Ruiz	European	Paralegal
2997	Joshua Ryan	American	Statistician IV
2998	Nicholas Cunningham	European	Human Resources Assistant III
2999	Joe Hanson	American	Biostatistician III

	Investment Advisor	...	Amount of Credit Cards	Credit Card Balance \
0	Victor Dean	...	1	484.5440
1	Jeremy Porter	...	1	2256.8777
2	Ernest Knight	...	2	4568.7438
3	Eric Shaw	...	2	4205.0010
4	Kevin Kim	...	1	3779.4880
...	...	...	...	...
2995	Daniel Carroll	...	1	649.8540
2996	Peter Castillo	...	1	1639.0350
2997	Steve Sanchez	...	1	2352.8448
2998	Juan Ramirez	...	2	3578.6088
2999	Peter Castillo	...	1	1494.6876

	Bank Loans	Bank Deposits	Checking Accounts	Saving Accounts \
0	7.762429e+05	1.485829e+06	6.036179e+05	607332.455240
1	1.270615e+06	6.414828e+05	2.295214e+05	344635.157402
2	1.052716e+06	1.033402e+06	6.526747e+05	203054.348179
3	1.211951e+05	1.048157e+06	1.048157e+06	234685.019326
4	1.048302e+06	4.877825e+05	4.466442e+05	128351.452320
...	...	...	...	...
2995	2.239351e+05	1.089957e+06	5.328679e+05	657849.619325
2996	5.959020e+05	1.368913e+05	5.658174e+04	93195.608103
2997	8.804930e+05	2.148609e+05	1.587261e+05	35539.152952
2998	2.686250e+05	7.426302e+05	4.046383e+05	56411.334112
2999	2.856407e+05	6.561766e+04	7.776908e+04	32371.380536

	Foreign Currency Account	Business Lending	Properties Owned \
0	12249.9584	1.134475e+06	1
1	61162.3089	2.000526e+06	1
2	79071.7794	5.481376e+05	1

3	57513.6520	1.148402e+06	0
4	30012.1360	1.674412e+06	0
...	...	...	...
2995	12947.3100	1.238860e+06	1
2996	23205.6900	2.771711e+05	1
2997	30291.8112	5.029472e+05	2
2998	6413.1444	1.538369e+06	3
2999	8992.3608	3.294125e+05	1

	Risk Weighting
0	2
1	3
2	3
3	4
4	3
...	...
2995	3
2996	2
2997	2
2998	1
2999	1

[3000 rows x 25 columns]

```
In [2]: df = pd.DataFrame(data)
print(df)
```

	Client ID	Name	Age	Sex	Location ID	Joined Bank \
0	PKR81288	Raymond Mills	24	Male	34324	2019-05-06
1	PKR65833	Julia Spencer	23	Male	42205	2001-12-10
2	PKR47499	Stephen Murray	27	Female	7314	2010-01-25
3	PKR72498	Virginia Garza	40	Male	34594	1993-03-28
4	PKR60181	Melissa Sanders	46	Female	41269	2012-07-20
...	...	...	...	...	...	...
2995	PKR66827	Earl Hall	82	Female	8760	2014-10-09
2996	PKR40556	Billy Williamson	44	Female	32837	2009-02-05
2997	PKR72414	Victor Black	70	Female	36088	2009-12-29
2998	PKR46652	Andrew Ford	56	Female	24871	2006-02-13
2999	PKR40216	Amy Nguyen	79	Female	38518	2005-12-08

	Banking Contact	Nationality	Occupation \
0	Anthony Torres	American	Safety Technician IV
1	Jonathan Hawkins	African	Software Consultant
2	Anthony Berry	European	Help Desk Operator
3	Steve Diaz	American	Geologist II
4	Shawn Long	American	Assistant Professor
...	...	...	...
2995	Joshua Bennett	American	Accounting Assistant III
2996	Dennis Ruiz	European	Paralegal
2997	Joshua Ryan	American	Statistician IV
2998	Nicholas Cunningham	European	Human Resources Assistant III
2999	Joe Hanson	American	Biostatistician III

	Investment Advisor	...	Amount of Credit Cards	Credit Card Balance \
0	Victor Dean	...	1	484.5440
1	Jeremy Porter	...	1	2256.8777
2	Ernest Knight	...	2	4568.7438
3	Eric Shaw	...	2	4205.0010
4	Kevin Kim	...	1	3779.4880
...	...	...	...	...
2995	Daniel Carroll	...	1	649.8540
2996	Peter Castillo	...	1	1639.0350
2997	Steve Sanchez	...	1	2352.8448
2998	Juan Ramirez	...	2	3578.6088
2999	Peter Castillo	...	1	1494.6876

	Bank Loans	Bank Deposits	Checking Accounts	Saving Accounts \
0	7.762429e+05	1.485829e+06	6.036179e+05	607332.455240
1	1.270615e+06	6.414828e+05	2.295214e+05	344635.157402
2	1.052716e+06	1.033402e+06	6.526747e+05	203054.348179
3	1.211951e+05	1.048157e+06	1.048157e+06	234685.019326
4	1.048302e+06	4.877825e+05	4.466442e+05	128351.452320
...	...	...	...	...
2995	2.239351e+05	1.089957e+06	5.328679e+05	657849.619325
2996	5.959020e+05	1.368913e+05	5.658174e+04	93195.608103
2997	8.804930e+05	2.148609e+05	1.587261e+05	35539.152952
2998	2.686250e+05	7.426302e+05	4.046383e+05	56411.334112
2999	2.856407e+05	6.561766e+04	7.776908e+04	32371.380536

	Foreign Currency Account	Business Lending	Properties Owned \
0	12249.9584	1.134475e+06	1
1	61162.3089	2.000526e+06	1
2	79071.7794	5.481376e+05	1

3	57513.6520	1.148402e+06	0
4	30012.1360	1.674412e+06	0
...	...	...	...
2995	12947.3100	1.238860e+06	1
2996	23205.6900	2.771711e+05	1
2997	30291.8112	5.029472e+05	2
2998	6413.1444	1.538369e+06	3
2999	8992.3608	3.294125e+05	1

	Risk Weighting
0	2
1	3
2	3
3	4
4	3
...	...
2995	3
2996	2
2997	2
2998	1
2999	1

[3000 rows x 25 columns]

In [3]: `df.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Client ID                            3000 non-null   str
1   Name                                3000 non-null   str
2   Age                                  3000 non-null   int64
3   Sex                                  3000 non-null   str
4   Location ID                          3000 non-null   int64
5   Joined Bank                          3000 non-null   datetime64[us]
6   Banking Contact                      3000 non-null   str
7   Nationality                          3000 non-null   str
8   Occupation                           3000 non-null   str
9   Investment Advisor                   3000 non-null   str
10  Fee Structure                        3000 non-null   str
11  Loyalty Classification                3000 non-null   str
12  Banking Relationship                  3000 non-null   str
13  Estimated Income                     3000 non-null   float64
14  Superannuation Savings                3000 non-null   float64
15  Amount of Credit Cards                3000 non-null   int64
16  Credit Card Balance                  3000 non-null   float64
17  Bank Loans                           3000 non-null   float64
18  Bank Deposits                        3000 non-null   float64
19  Checking Accounts                    3000 non-null   float64
20  Saving Accounts                      3000 non-null   float64
21  Foreign Currency Account              3000 non-null   float64
22  Business Lending                     3000 non-null   float64
23  Properties Owned                     3000 non-null   int64
24  Risk Weighting                       3000 non-null   int64
dtypes: datetime64[us](1), float64(9), int64(5), str(10)
memory usage: 586.1 KB

```

```
In [4]: df.isnull().sum()
```

```
Out[4]: Client ID      0
        Name          0
        Age           0
        Sex           0
        Location ID   0
        Joined Bank   0
        Banking Contact 0
        Nationality   0
        Occupation    0
        Investment Advisor 0
        Fee Structure 0
        Loyalty Classification 0
        Banking Relationship 0
        Estimated Income 0
        Superannuation Savings 0
        Amount of Credit Cards 0
        Credit Card Balance 0
        Bank Loans    0
        Bank Deposits 0
        Checking Accounts 0
        Saving Accounts 0
        Foreign Currency Account 0
        Business Lending 0
        Properties Owned 0
        Risk Weighting 0
        dtype: int64
```

```
In [6]: missing_percentage = (df.isnull().sum() / len(df)) * 100
        print(missing_percentage)
```

```
Client ID      0.0
Name          0.0
Age           0.0
Sex           0.0
Location ID   0.0
Joined Bank   0.0
Banking Contact 0.0
Nationality   0.0
Occupation    0.0
Investment Advisor 0.0
Fee Structure  0.0
Loyalty Classification 0.0
Banking Relationship 0.0
Estimated Income 0.0
Superannuation Savings 0.0
Amount of Credit Cards 0.0
Credit Card Balance 0.0
Bank Loans    0.0
Bank Deposits 0.0
Checking Accounts 0.0
Saving Accounts 0.0
Foreign Currency Account 0.0
Business Lending 0.0
Properties Owned 0.0
Risk Weighting 0.0
dtype: float64
```

```
In [14]: df.duplicated().sum()
```

```
Out[14]: np.int64(0)
```

```
In [15]: df["Client ID"].duplicated().sum()
```

```
Out[15]: np.int64(60)
```

```
In [18]: df[df["Client ID"].duplicated(keep = False)].sort_values("Client ID")
```

```
Out[18]:
```

	Client ID	Name	Age	Sex	Location ID	Joined Bank	Banking Contact	Nationality	Occ
1545	PKR15088	Margaret Coleman	42	Female	8628	2010-12-25	Keith Griffin	American	Adm
740	PKR15088	Betty Davis	56	Male	12337	2013-03-14	Jesse Evans	European	M
378	PKR15955	Susan Dunn	20	Female	2035	2011-12-08	Jason Duncan	Asian	Pro A
1081	PKR15955	Victor Williams	21	Female	26992	2015-05-29	Mark Montgomery	African	As
995	PKR17019	Keith Fuller	70	Male	14000	2014-10-25	Jesse Evans	European	R
...	...	...	...	...	...	...	...	...	
2926	PKR94062	Kathryn Graham	76	Male	29597	2014-12-26	Roger Alexander	American	
2901	PKR96193	Adam Murphy	54	Male	21076	2009-07-28	Samuel Fowler	European	Pro
2013	PKR96193	Lori Hawkins	44	Male	34985	2009-08-29	Joshua Little	American	Repre
1738	PKR97314	John Davis	80	Female	38563	1998-01-09	Paul Holmes	American	
2775	PKR97314	Angela Green	51	Male	1339	1994-03-12	Anthony Berry	European	Admi As

119 rows × 25 columns



```
In [20]: df["Client ID"].value_counts()
```

```
Out[20]: Client ID
PKR48103    3
PKR35589    2
PKR54314    2
PKR74305    2
PKR26283    2
..
PKR66827    1
PKR40556    1
PKR72414    1
PKR46652    1
PKR40216    1
Name: count, Length: 2940, dtype: int64
```

```
In [21]: df["Client ID"].value_counts().head(10)
```

```
Out[21]: Client ID
PKR48103    3
PKR35589    2
PKR54314    2
PKR74305    2
PKR26283    2
PKR49489    2
PKR50658    2
PKR31656    2
PKR67912    2
PKR94055    2
Name: count, dtype: int64
```

```
In [22]: df[df["Client ID"] == "PKR48103"]
```

Out[22]:

	Client ID	Name	Age	Sex	Location ID	Joined Bank	Banking Contact	Nationality	
1133	PKR48103	Pamela Simmons	50	Female	688	2015-10-04	Nicholas Simmons	European	Budget
1244	PKR48103	George Jones	71	Male	17485	2004-07-13	Paul Holmes	European	Staff
1803	PKR48103	Douglas Cole	73	Female	35102	2006-11-02	Mark Montgomery	Australian	

3 rows × 25 columns

```
In [24]: summary = df.describe()
summary
```

Out[24]:

	Age	Location ID	Joined Bank	Estimated Income	Superannuation Savings	Amount Cr
count	3000.000000	3000.000000	3000	3000.000000	3000.000000	3000.000000
mean	51.039667	21563.323000	2006-01-27 22:29:45.600000	171305.034094	25531.599673	1.463000
min	17.000000	12.000000	1992-05-23 00:00:00	15919.475400	1482.030000	1.000000
25%	34.000000	10803.500000	1998-12-30 18:00:00	82906.594300	12513.775000	1.000000
50%	51.000000	21129.500000	2006-02-11 00:00:00	142313.479600	22357.355000	1.000000
75%	69.000000	32054.500000	2013-03-06 00:00:00	242290.306650	35464.740000	2.000000
max	85.000000	43369.000000	2019-09-12 00:00:00	522330.259200	75963.900000	3.000000
std	19.854760	12462.273017	NaN	111935.808260	16259.950770	0.676000

In [30]: `df.describe(include = "string")`

Out[30]:

	Client ID	Name	Sex	Banking Contact	Nationality	Occupation	Investment Advisor	Struc
count	3000	3000	3000	3000	3000	3000	3000	3000
unique	2940	2913	2	49	5	195	22	22
top	PKR48103	Raymond Mills	Female	Ernest Rivera	European	Associate Professor	Eugene Cunningham	1
freq	3	2	1512	77	1309	28	155	155

In [31]: `total_customers = df.shape[0]`
`print("total_customers", total_customers)`

total_customers 3000

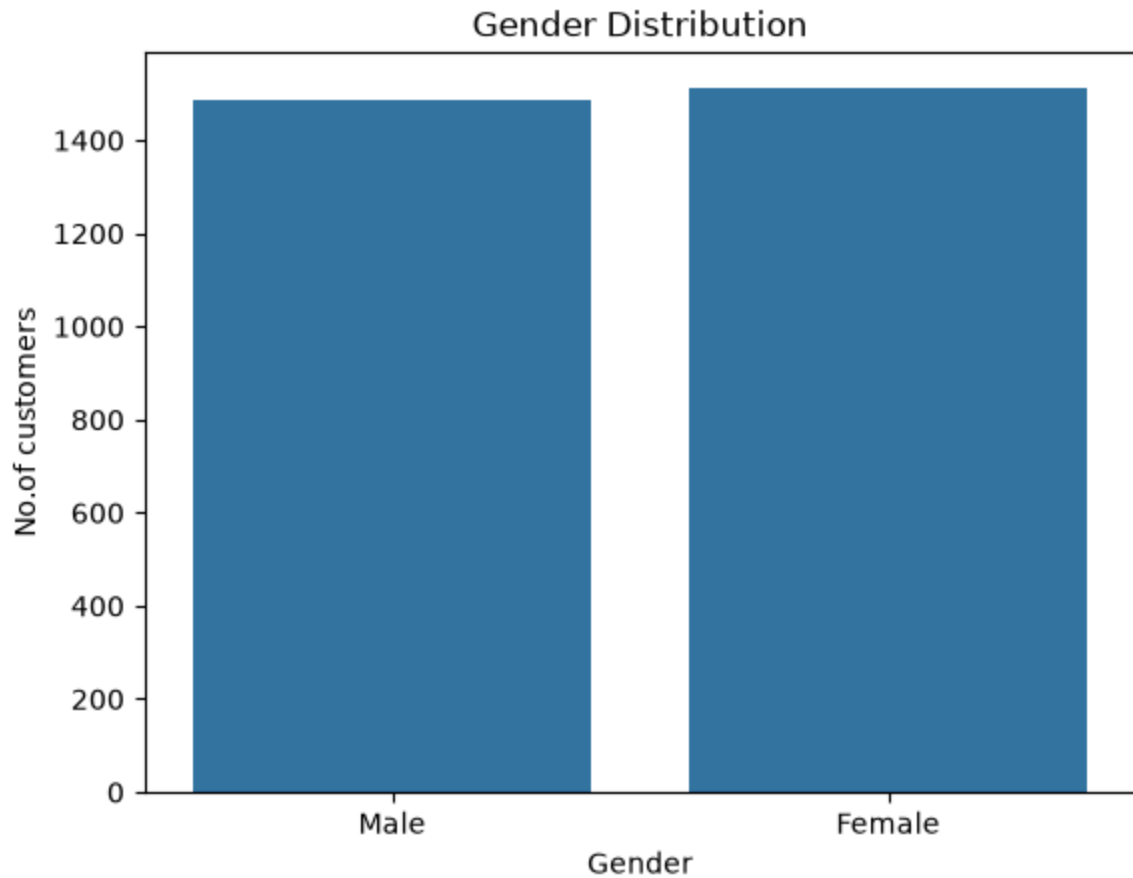
In [33]: `df["Sex"].value_counts()`

Out[33]: Sex
Female 1512
Male 1488
Name: count, dtype: int64

In [34]: `(df["Sex"].value_counts(normalize=True) * 100).round(2)`

```
Out[34]: Sex
        Female    50.4
        Male      49.6
        Name: proportion, dtype: float64
```

```
In [35]: sns.countplot(x="Sex", data = df)
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("No.of customers")
plt.show()
```



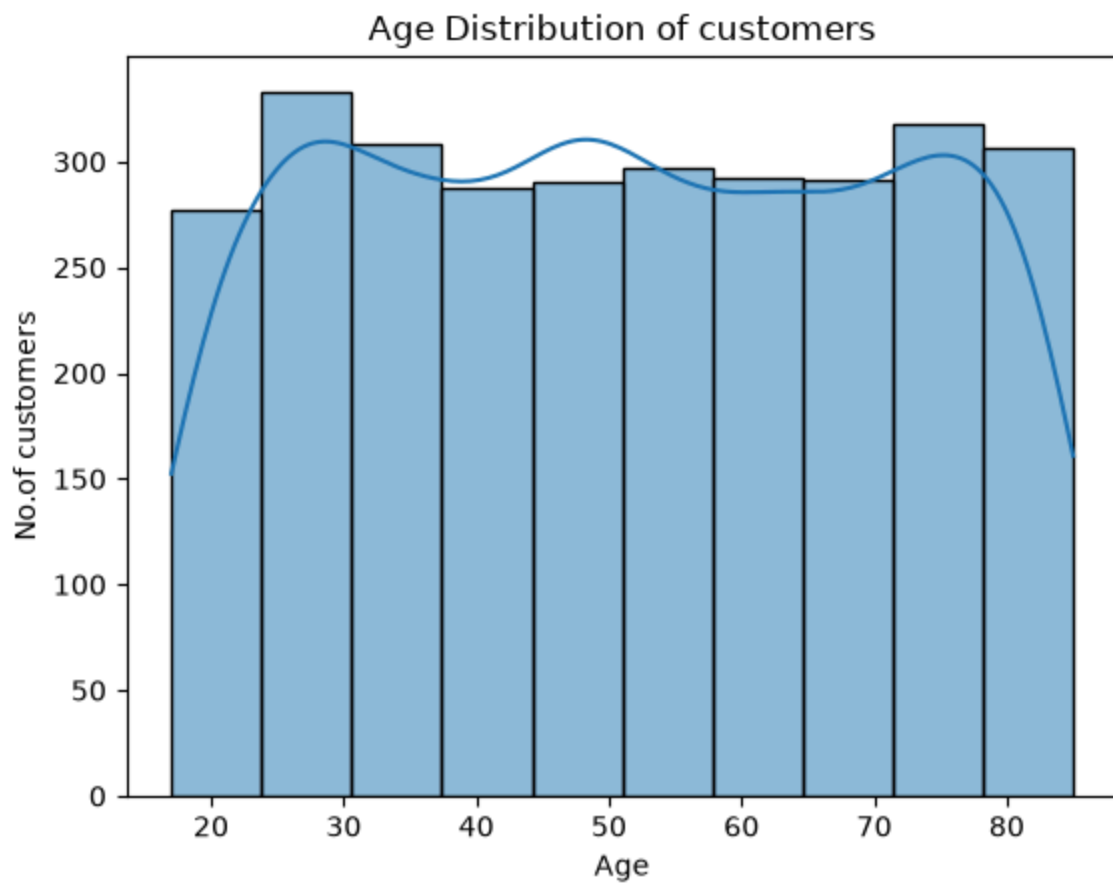
```
In [36]: df["Age"].mean()
```

```
Out[36]: np.float64(51.039666666666667)
```

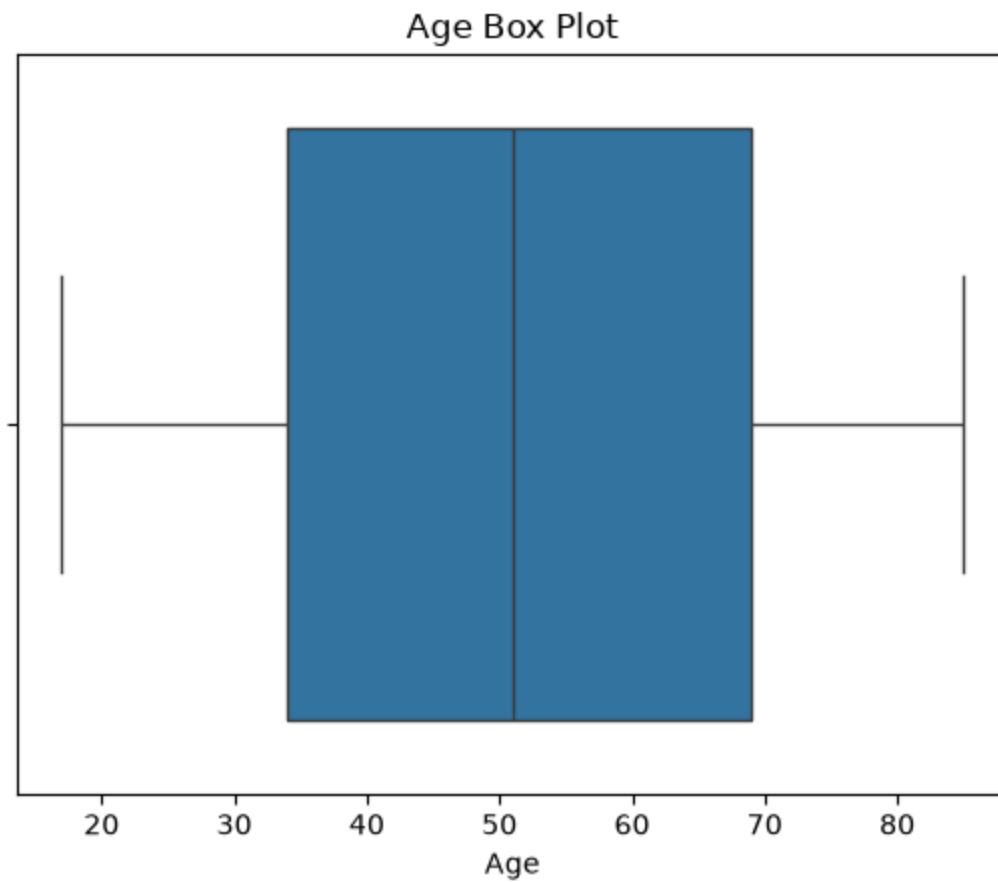
```
In [37]: print("Minimum Age:", df["Age"].min())
print("Maximum Age:", df["Age"].max())
```

```
Minimum Age: 17
Maximum Age: 85
```

```
In [38]: sns.histplot(x=df["Age"], bins=10, kde=True)
plt.title("Age Distribution of customers")
plt.xlabel("Age")
plt.ylabel("No.of customers")
plt.show()
```



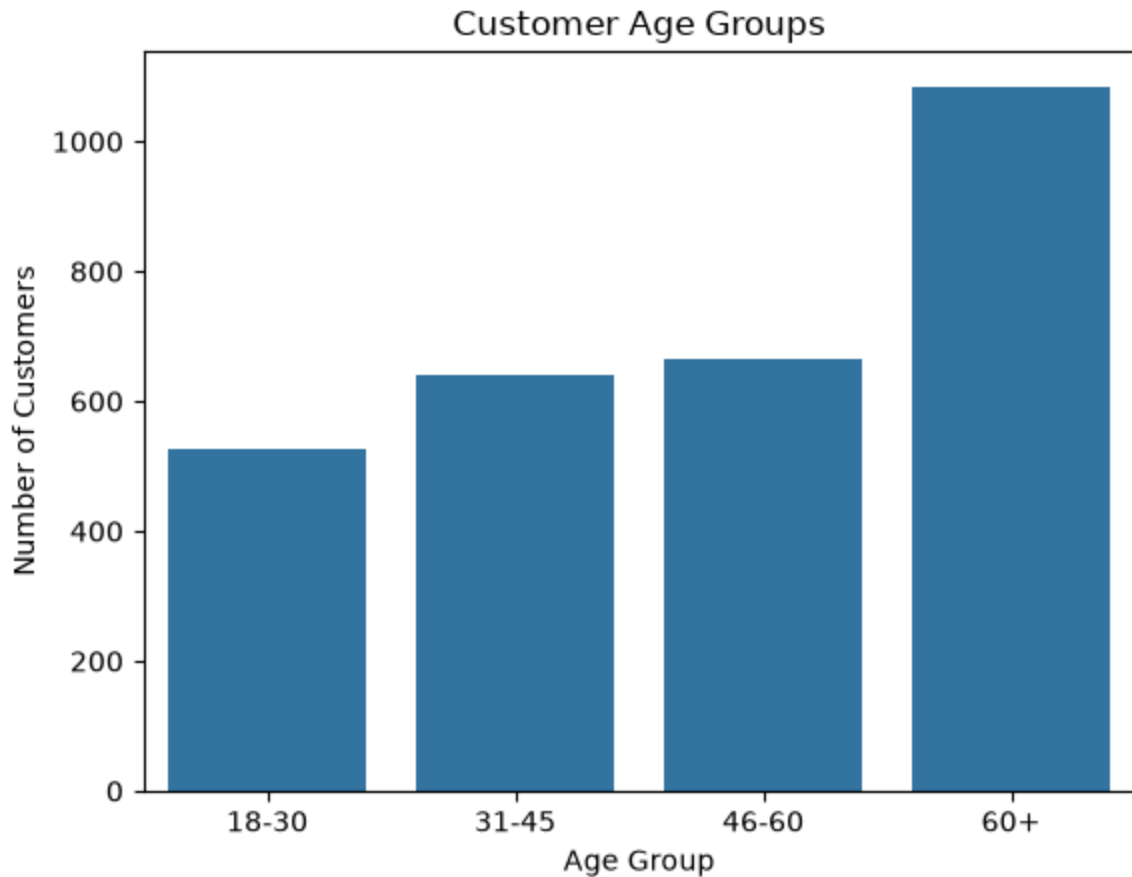
```
In [39]: sns.boxplot(x=df["Age"])  
plt.title("Age Box Plot")  
plt.show()
```



```
In [40]: age_groups = pd.cut(df["Age"], bins=[18,30,45,60,100], labels=["18-30", "31-45", "46-  
age_groups.value_counts()
```

```
Out[40]: Age  
60+      1084  
46-60     666  
31-45     640  
18-30     526  
Name: count, dtype: int64
```

```
In [41]: sns.countplot(x=age_groups)  
plt.title("Customer Age Groups")  
plt.xlabel("Age Group")  
plt.ylabel("Number of Customers")  
plt.show()
```



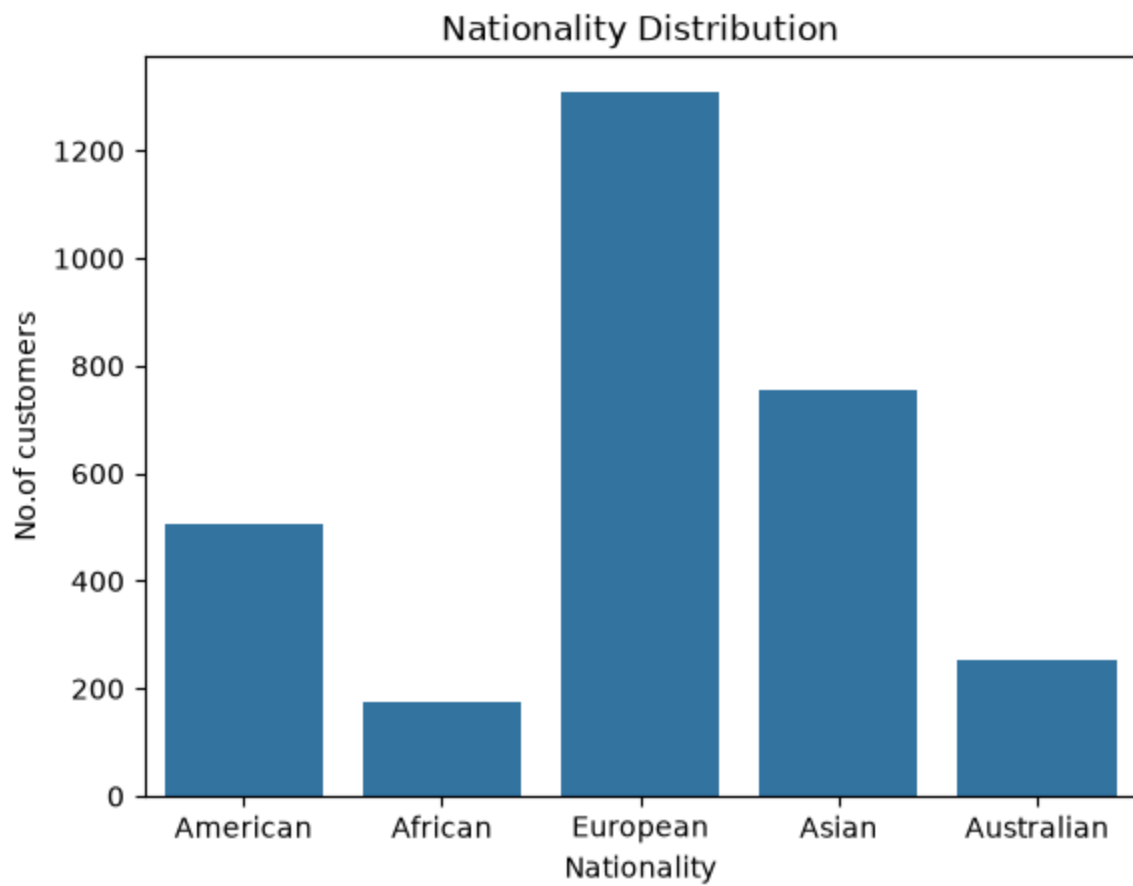
```
In [42]: df["Nationality"].value_counts()
```

```
Out[42]: Nationality
European    1309
Asian       754
American    507
Australian   254
African     176
Name: count, dtype: int64
```

```
In [45]: percentage = (df["Nationality"].value_counts(normalize=True) * 100).round(2)
percentage
```

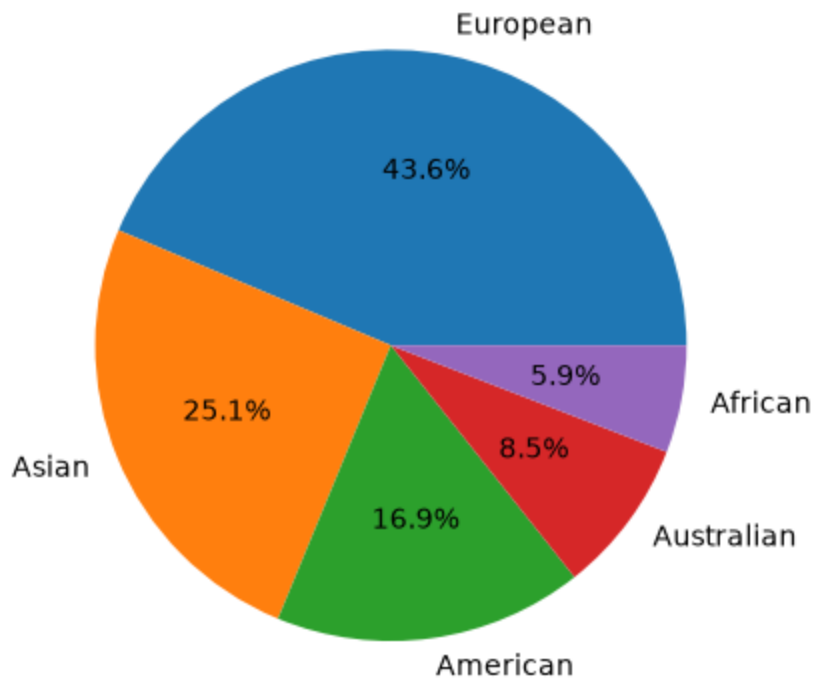
```
Out[45]: Nationality
European    43.63
Asian       25.13
American    16.90
Australian   8.47
African      5.87
Name: proportion, dtype: float64
```

```
In [44]: sns.countplot(x="Nationality", data = df)
plt.title("Nationality Distribution")
plt.xlabel("Nationality")
plt.ylabel("No.of customers")
plt.show()
```



```
In [48]: df["Nationality"].value_counts().plot(kind="pie", autopct="%1.1f%")  
plt.title("Nationality Percentage Distribution")  
plt.ylabel("")  
plt.show()
```

Nationality Percentage Distribution



```
In [49]: df["Occupation"].unique()
```

```
Out[49]: <StringArray>
[      'Safety Technician IV',      'Software Consultant',
       'Help Desk Operator',      'Geologist II',
       'Assistant Professor',      'Help Desk Technician',
       'Account Coordinator', 'Automation Specialist II',
       'Software Test Engineer II', 'Geologist III',
       ...
       'Biostatistician II',      'Senior Cost Accountant',
       'GIS Technical Architect',      'General Manager',
       'Nurse Practitioner',      'Recruiting Manager',
       'Safety Technician II',      'Sales Representative',
       'Automation Specialist I',      'Senior Sales Associate']
Length: 195, dtype: str
```

```
In [50]: df["Occupation"].nunique()
```

```
Out[50]: 195
```

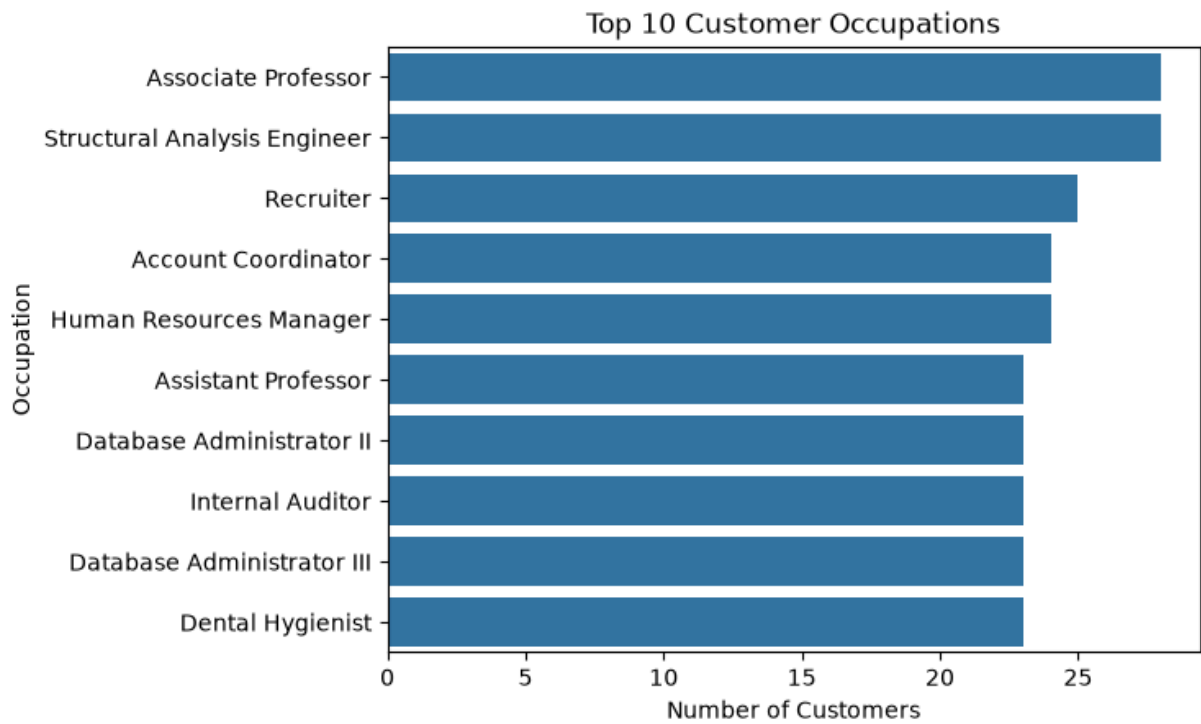
```
In [51]: df["Occupation"].value_counts()
```

```
Out[51]: Occupation
Associate Professor      28
Structural Analysis Engineer  28
Recruiter                25
Account Coordinator      24
Human Resources Manager   24
..
Office Assistant IV      8
Automation Specialist I   7
Computer Systems Analyst I 6
Developer III            5
Senior Sales Associate     4
Name: count, Length: 195, dtype: int64
```

```
In [52]: df["Occupation"].value_counts().head(10)
```

```
Out[52]: Occupation
Associate Professor      28
Structural Analysis Engineer  28
Recruiter                25
Account Coordinator      24
Human Resources Manager   24
Assistant Professor       23
Database Administrator II  23
Internal Auditor          23
Database Administrator III 23
Dental Hygienist          23
Name: count, dtype: int64
```

```
In [60]: top10 = df["Occupation"].value_counts().head(10)
sns.barplot(x=top10.values, y=top10.index)
plt.title("Top 10 Customer Occupations")
plt.xlabel("Number of Customers")
plt.ylabel("Occupation")
plt.show()
```



```
In [53]: df["Banking Relationship"].unique()
```

```
Out[53]: <StringArray>  
['Retail', 'Institutional', 'Private Bank', 'Commercial']  
Length: 4, dtype: str
```

```
In [54]: df["Banking Relationship"].nunique()
```

```
Out[54]: 4
```

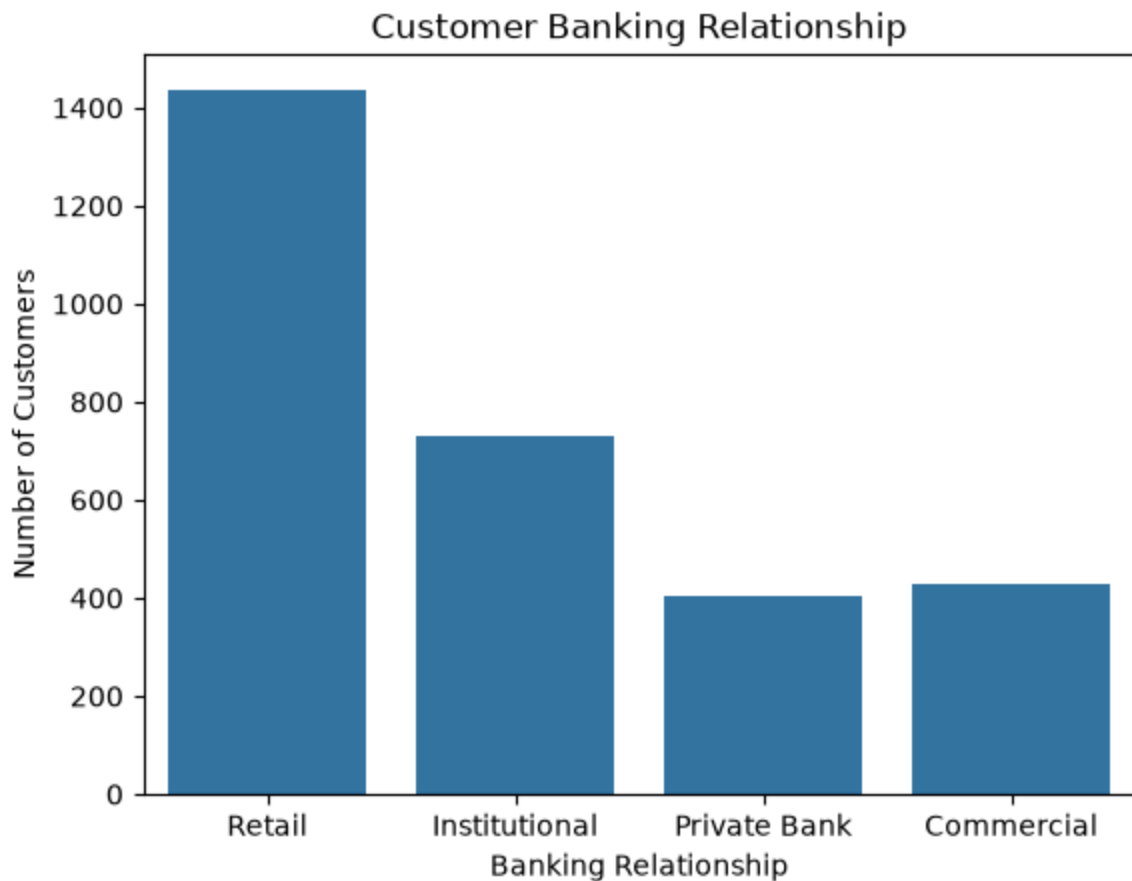
```
In [56]: df["Banking Relationship"].value_counts()
```

```
Out[56]: Banking Relationship  
Retail          1436  
Institutional    729  
Commercial       430  
Private Bank     405  
Name: count, dtype: int64
```

```
In [57]: (df["Banking Relationship"].value_counts(normalize=True) * 100).round(2)
```

```
Out[57]: Banking Relationship  
Retail          47.87  
Institutional   24.30  
Commercial      14.33  
Private Bank    13.50  
Name: proportion, dtype: float64
```

```
In [61]: sns.countplot(x="Banking Relationship", data=df)
plt.title("Customer Banking Relationship")
plt.xlabel("Banking Relationship")
plt.ylabel("Number of Customers")
plt.show()
```



```
In [62]: df["Estimated Income"].mean()
```

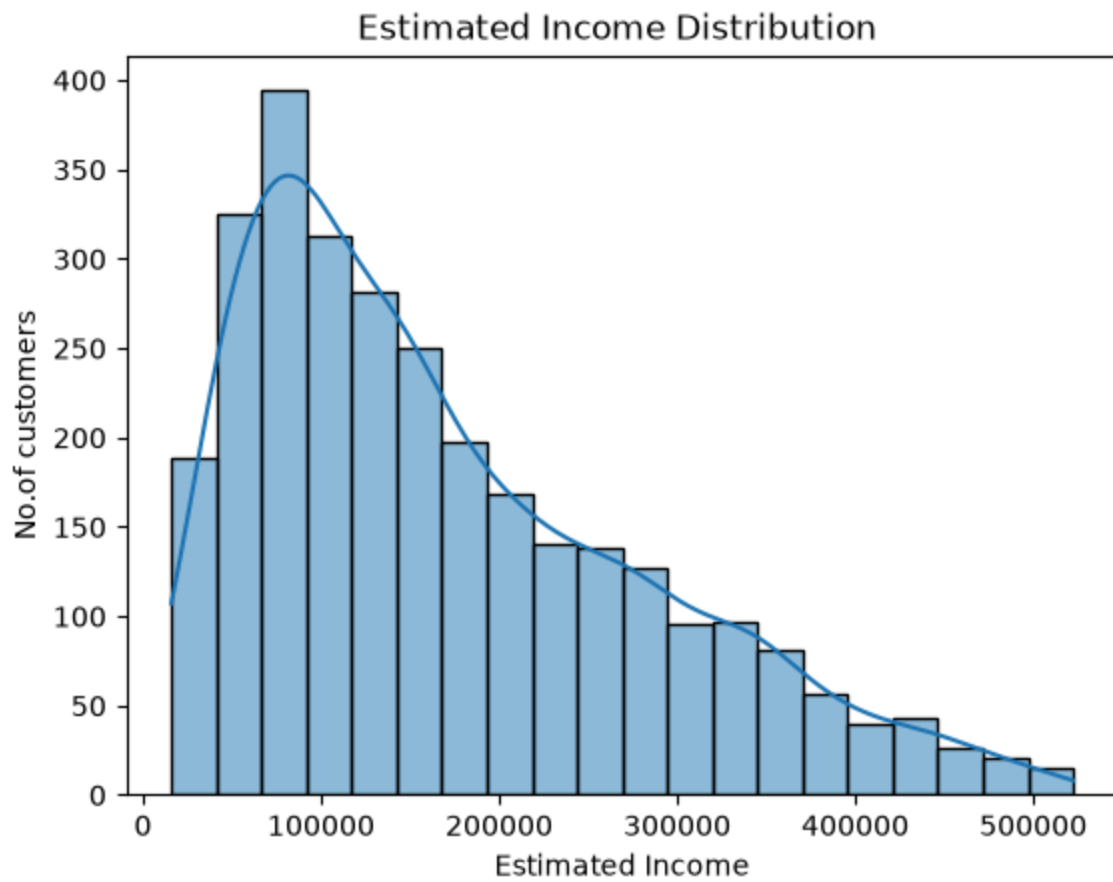
```
Out[62]: np.float64(171305.03409373335)
```

```
In [63]: print("Minimum Income:", df["Estimated Income"].min())
print("Maximum Income:", df["Estimated Income"].max())
```

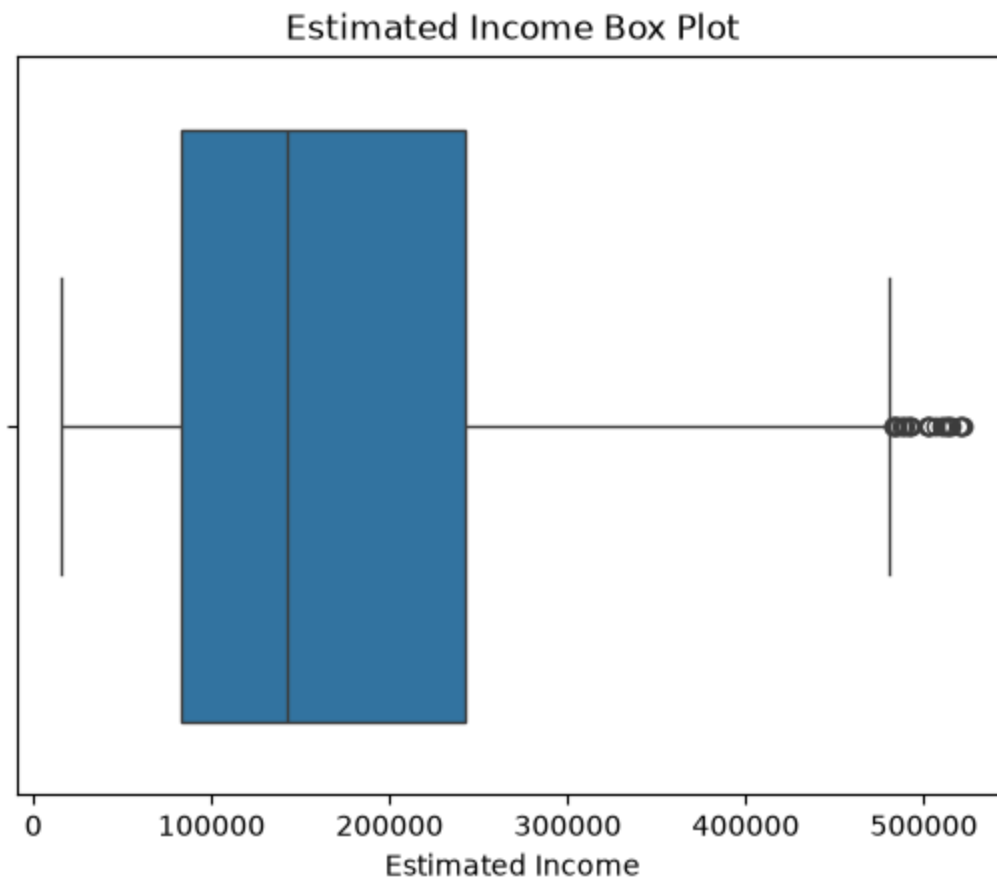
```
Minimum Income: 15919.475400000001
```

```
Maximum Income: 522330.25920000003
```

```
In [64]: sns.histplot(x=df["Estimated Income"], bins=20, kde=True)
plt.title("Estimated Income Distribution ")
plt.xlabel("Estimated Income")
plt.ylabel("No.of customers")
plt.show()
```



```
In [77]: sns.boxplot(x=df["Estimated Income"])  
plt.title("Estimated Income Box Plot")  
plt.show()
```



```
In [66]: df.nlargest(10, "Estimated Income")[["Client ID", "Name", "Estimated Income"]]
```

```
Out[66]:
```

	Client ID	Name	Estimated Income
1279	PKR86610	Tammy Pierce	522330.2592
1189	PKR70814	Theresa Cooper	521159.9844
1093	PKR48208	Jerry Mitchell	520328.3475
1017	PKR17839	Larry Gibson	520192.4352
1476	PKR47563	Alice Marshall	514809.5744
2465	PKR85022	Carolyn Wagner	514507.7925
1819	PKR91720	Harry Gibson	513292.8768
1537	PKR96715	Gary King	512987.8047
2411	PKR62941	Joshua Gibson	511043.9280
1179	PKR96098	Eric Sullivan	510066.3022

```
In [67]: df.nsmallest(10, "Estimated Income")[["Client ID", "Name", "Estimated Income"]]
```

Out[67]:

	Client ID	Name	Estimated Income
1704	PKR29286	Carolyn Martinez	15919.4754
2713	PKR22264	James Burton	16653.9672
1811	PKR29309	Joan Duncan	17226.8544
1113	PKR18504	Stephen Kim	17367.2184
481	PKR90141	Helen Evans	17723.1250
2082	PKR45633	Matthew Ramos	17871.5625
334	PKR87900	Joan Martinez	17987.5053
2635	PKR48222	Walter Griffin	18687.5037
2032	PKR49496	Martin Martinez	18691.1361
2373	PKR38948	Donald Bennett	19083.6096

```
In [69]: df["Bank Deposits"].sum()
```

Out[69]: np.float64(2014680581.5270572)

```
In [70]: df["Bank Deposits"].mean()
```

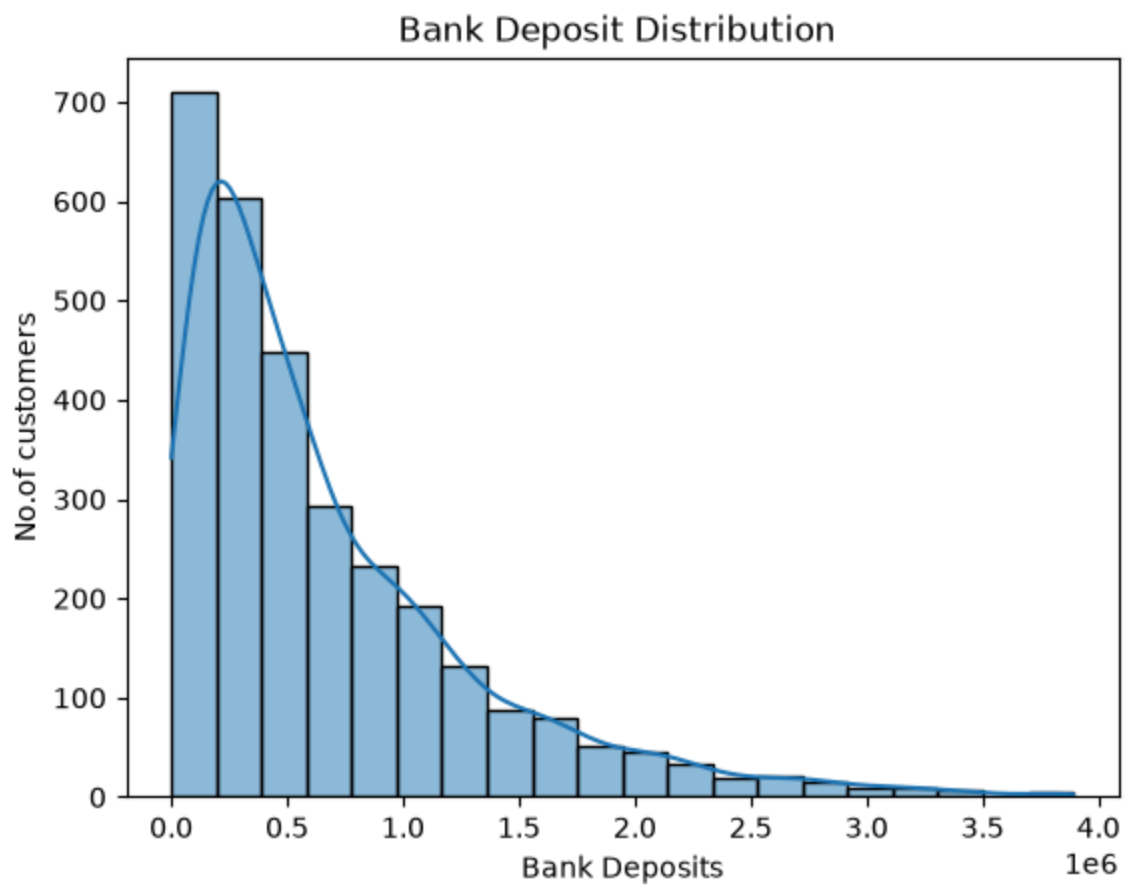
Out[70]: np.float64(671560.1938423524)

```
In [73]: print("Highest Deposit:", df["Bank Deposits"].max())
print("Lowest Deposit:", df["Bank Deposits"].min())
```

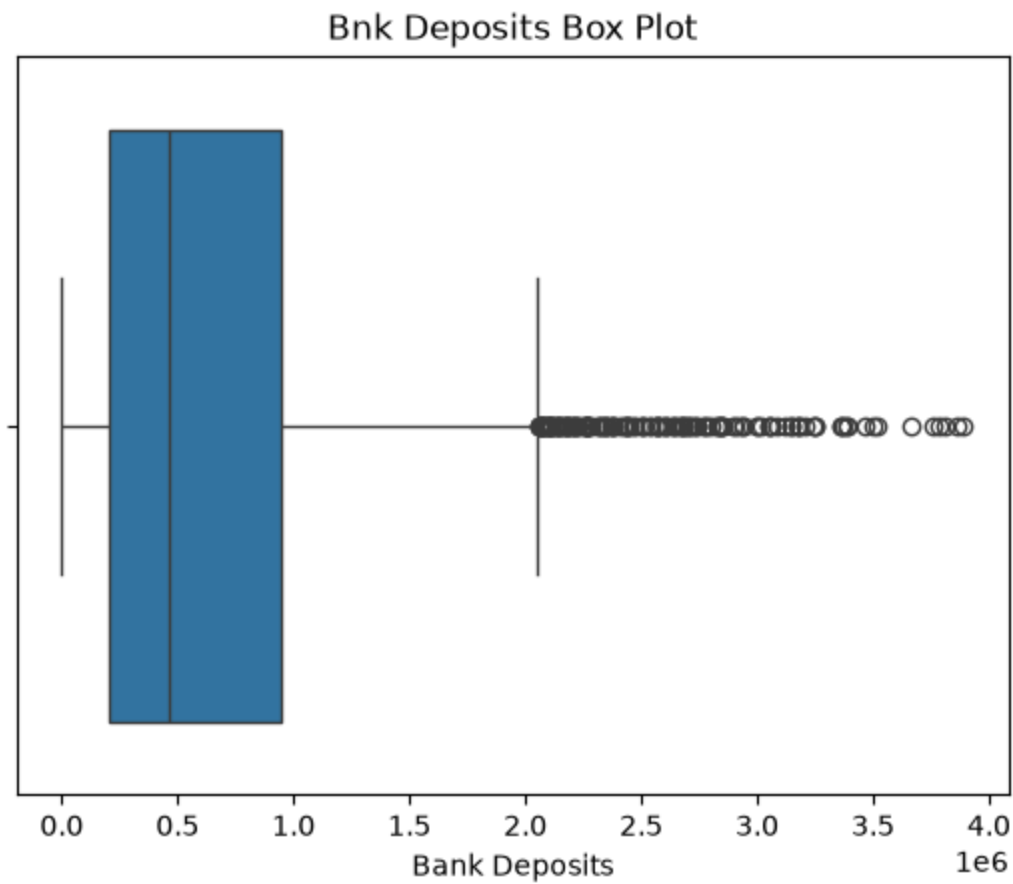
Highest Deposit: 3890598.08458

Lowest Deposit: 0.0

```
In [76]: sns.histplot(x=df["Bank Deposits"], bins=20, kde=True)
plt.title("Bank Deposit Distribution ")
plt.xlabel("Bank Deposits")
plt.ylabel("No.of customers")
plt.show()
```



```
In [78]: sns.boxplot(x=df["Bank Deposits"])
plt.title("Bnk Deposits Box Plot")
plt.show()
```



```
In [79]: df.nlargest(10, "Bank Deposits")[["Client ID", "Name", "Bank Deposits"]]
```

```
Out[79]:
```

	Client ID	Name	Bank Deposits
640	PKR68515	Ronald Reed	3.890598e+06
2307	PKR81510	Harry Burns	3.863355e+06
1132	PKR70565	Craig Hansen	3.808500e+06
1819	PKR91720	Harry Gibson	3.784995e+06
1643	PKR47511	Johnny Patterson	3.760505e+06
2109	PKR53236	John Frazier	3.667387e+06
952	PKR52376	Scott Hart	3.515973e+06
376	PKR32547	Wayne Hart	3.498984e+06
2594	PKR86149	Martin Marshall	3.466323e+06
2567	PKR22373	Clarence Hart	3.386869e+06

```
In [80]: df.nsmallest(10, "Bank Deposits")[["Client ID", "Name", "Bank Deposits"]]
```

Out[80]:

	Client ID	Name	Bank Deposits
63	PKR93407	Karen Morrison	0.0
213	PKR86593	Gregory Johnston	0.0
296	PKR35099	Thomas Hanson	0.0
321	PKR41393	Barbara Smith	0.0
446	PKR66550	Patricia George	0.0
501	PKR54434	Brian Morgan	0.0
537	PKR96395	Joshua Pierce	0.0
604	PKR55634	Scott Jones	0.0
670	PKR78396	Anthony Howard	0.0
705	PKR22467	Joseph Austin	0.0

In [81]: `df["Bank Loans"].sum()`

Out[81]: `np.float64(1774158465.7594)`

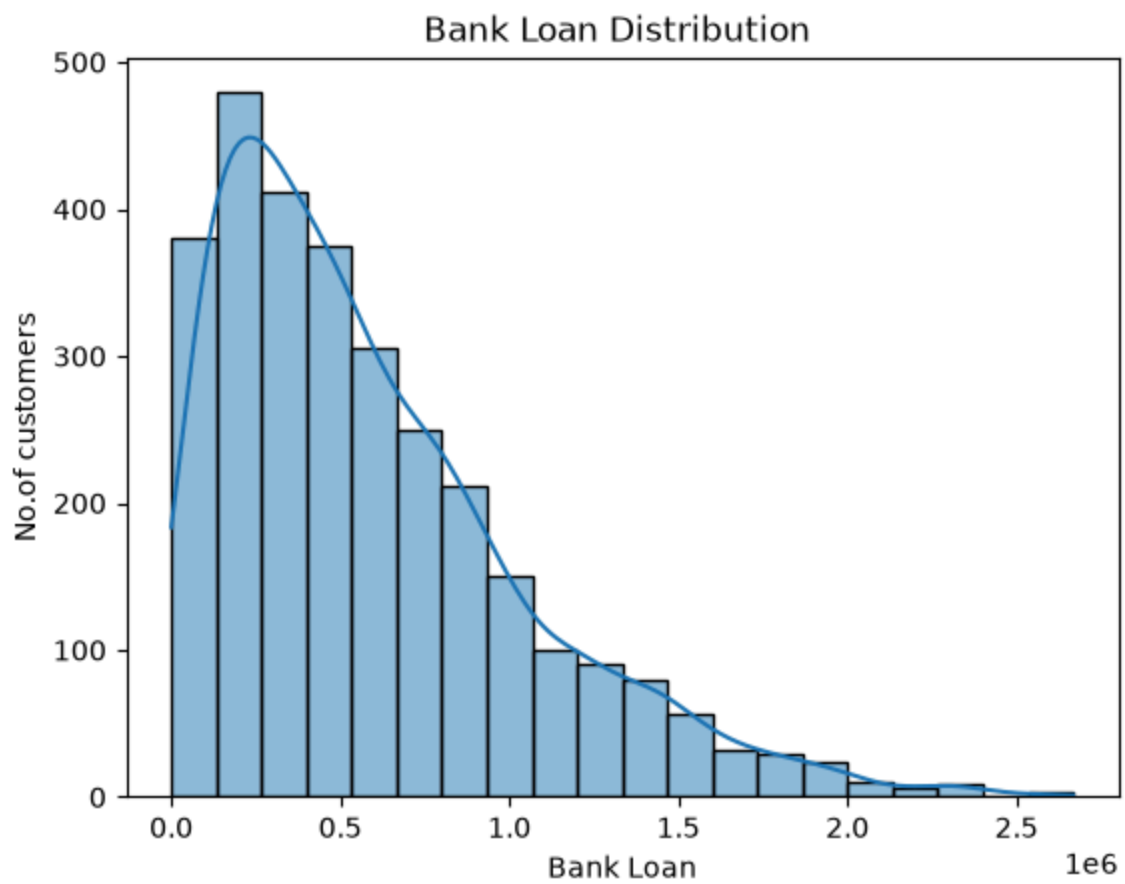
In [82]: `df["Bank Loans"].mean()`

Out[82]: `np.float64(591386.1552531333)`

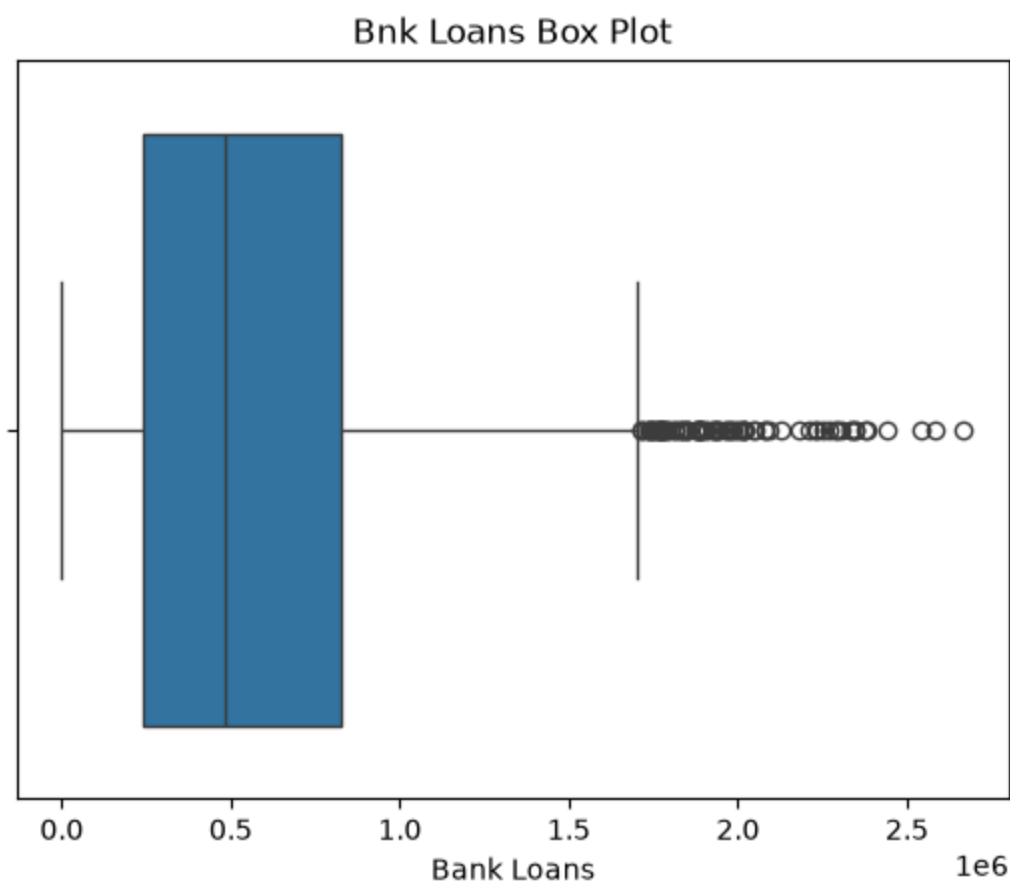
In [83]: `print("Highest Loan:", df["Bank Loans"].max())`
`print("Lowest Loan:", df["Bank Loans"].min())`

Highest Loan: 2667556.661
Lowest Loan: 0.0

In [84]: `sns.histplot(x=df["Bank Loans"], bins=20, kde=True)`
`plt.title("Bank Loan Distribution ")`
`plt.xlabel("Bank Loan")`
`plt.ylabel("No.of customers")`
`plt.show()`



```
In [85]: sns.boxplot(x=df["Bank Loans"])  
plt.title("Bnk Loans Box Plot")  
plt.show()
```



```
In [86]: df.nlargest(10, "Bank Loans")[["Client ID", "Name", "Bank Loans"]]
```

```
Out[86]:
```

	Client ID	Name	Bank Loans
2704	PKR23172	Ryan Mitchell	2.667557e+06
2097	PKR22626	Terry Chavez	2.584311e+06
2628	PKR99978	Russell Wagner	2.538730e+06
1334	PKR99607	Sean Pierce	2.439164e+06
760	PKR70050	Ralph Sullivan	2.381138e+06
1595	PKR19422	Sean Harrison	2.374318e+06
1132	PKR70565	Craig Hansen	2.349148e+06
2802	PKR31640	Larry Cooper	2.348592e+06
952	PKR52376	Scott Hart	2.333488e+06
2772	PKR63638	Daniel Flores	2.331797e+06

```
In [87]: df.nsmallest(10, "Bank Loans")[["Client ID", "Name", "Bank Loans"]]
```

Out[87]:

	Client ID	Name	Bank Loans
40	PKR60970	Philip Fisher	0.0
100	PKR80929	Mary Austin	0.0
266	PKR33193	Scott Ward	0.0
409	PKR65488	Bonnie Simpson	0.0
422	PKR72516	Beverly Stewart	0.0
453	PKR74531	Edward Mitchell	0.0
725	PKR99321	Mary Jenkins	0.0
925	PKR90914	Scott Phillips	0.0
1021	PKR73620	Robert Stewart	0.0
1061	PKR34623	Joyce Sanders	0.0

```
In [89]: df["Credit Card Balance"].sum()
```

Out[89]: np.float64(9528620.3398)

```
In [90]: df["Credit Card Balance"].mean()
```

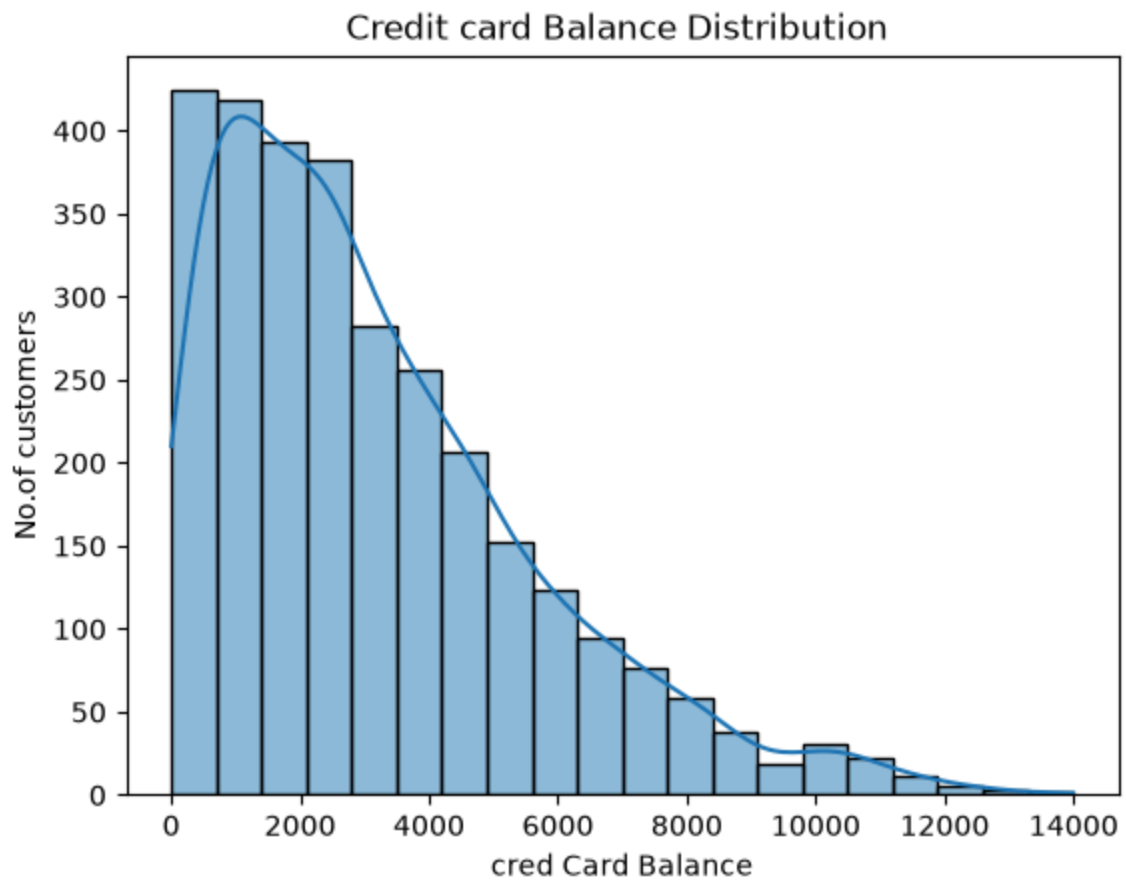
Out[90]: np.float64(3176.2067799333336)

```
In [91]: print("Highest Balance:", df["Credit Card Balance"].max())
print("Lowest Balance:", df["Credit Card Balance"].min())
```

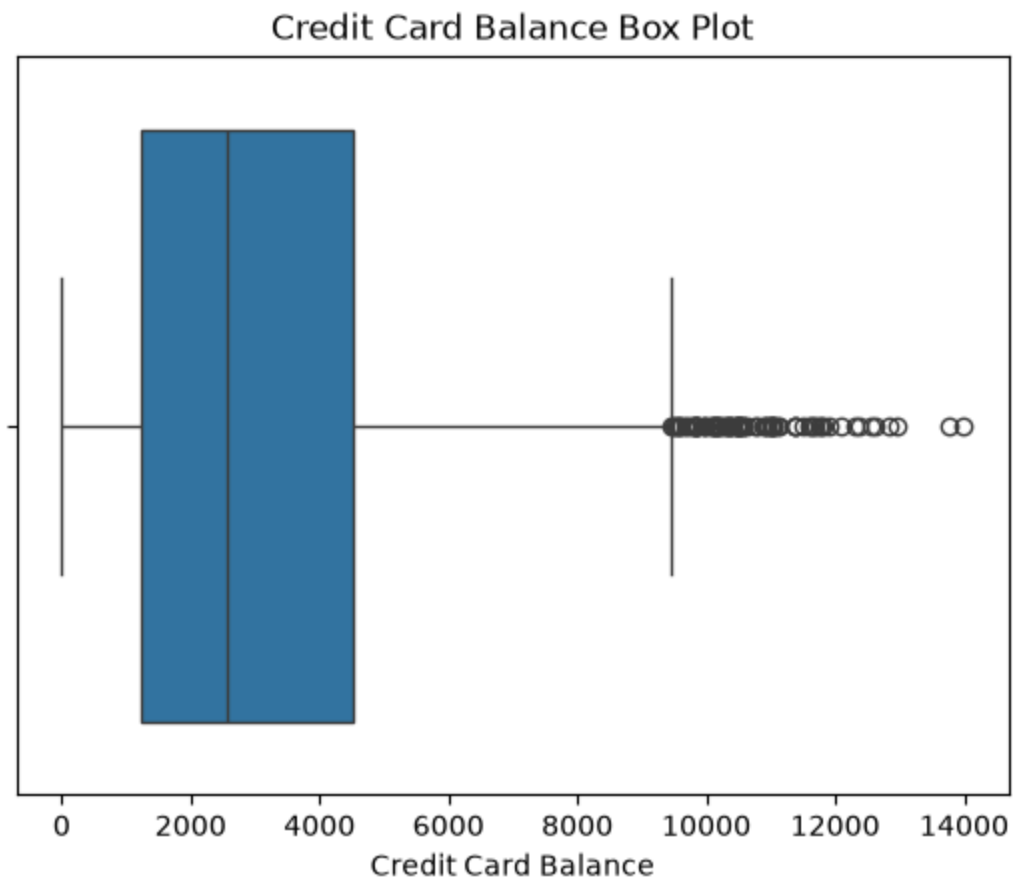
Highest Balance: 13991.993999999999

Lowest Balance: 1.1748

```
In [92]: sns.histplot(x=df["Credit Card Balance"], bins=20, kde=True)
plt.title("Credit card Balance Distribution ")
plt.xlabel("cred Card Balance")
plt.ylabel("No.of customers")
plt.show()
```



```
In [94]: sns.boxplot(x=df["Credit Card Balance"])  
plt.title("Credit Card Balance Box Plot")  
plt.show()
```



In [95]: `df.nlargest(10, "Credit Card Balance")[["Client ID", "Name", "Credit Card Balance"]]`

Out[95]:

	Client ID	Name	Credit Card Balance
1192	PKR23636	Brandon Patterson	13991.9940
2628	PKR99978	Russell Wagner	13749.8250
2780	PKR99855	Christopher Cooper	12970.0032
1504	PKR89040	Earl Hart	12841.8150
2964	PKR71997	Robert Gibson	12607.6800
2567	PKR22373	Clarence Hart	12549.8230
363	PKR41835	Joseph Burns	12379.0140
2703	PKR35208	Earl Wagner	12294.4345
1579	PKR76102	Jose King	12102.4620
1138	PKR72949	Robert Harrison	11910.9900

In [96]: `df.nsmallest(10, "Credit Card Balance")[["Client ID", "Name", "Credit Card Balance"]]`

Out[96]:

	Client ID	Name	Credit Card Balance
266	PKR33193	Scott Ward	1.1748
2203	PKR35253	Denise Hamilton	4.2484
1281	PKR90885	Ronald Austin	4.3560
190	PKR34869	Shawn Lane	5.7420
1232	PKR59974	Joshua Phillips	5.9584
2383	PKR89040	Harold Williams	6.9768
227	PKR38441	Stephen Stewart	7.1148
420	PKR45574	Janice Foster	7.4970
2633	PKR52005	Scott Franklin	7.6440
40	PKR60970	Philip Fisher	9.5526

```
In [97]: df["Properties Owned"].mean()
```

Out[97]: np.float64(1.5186666666666666)

```
In [98]: print("Minimum Properties:", df["Properties Owned"].min())
print("Maximum Properties:", df["Properties Owned"].max())
```

Minimum Properties: 0

Maximum Properties: 3

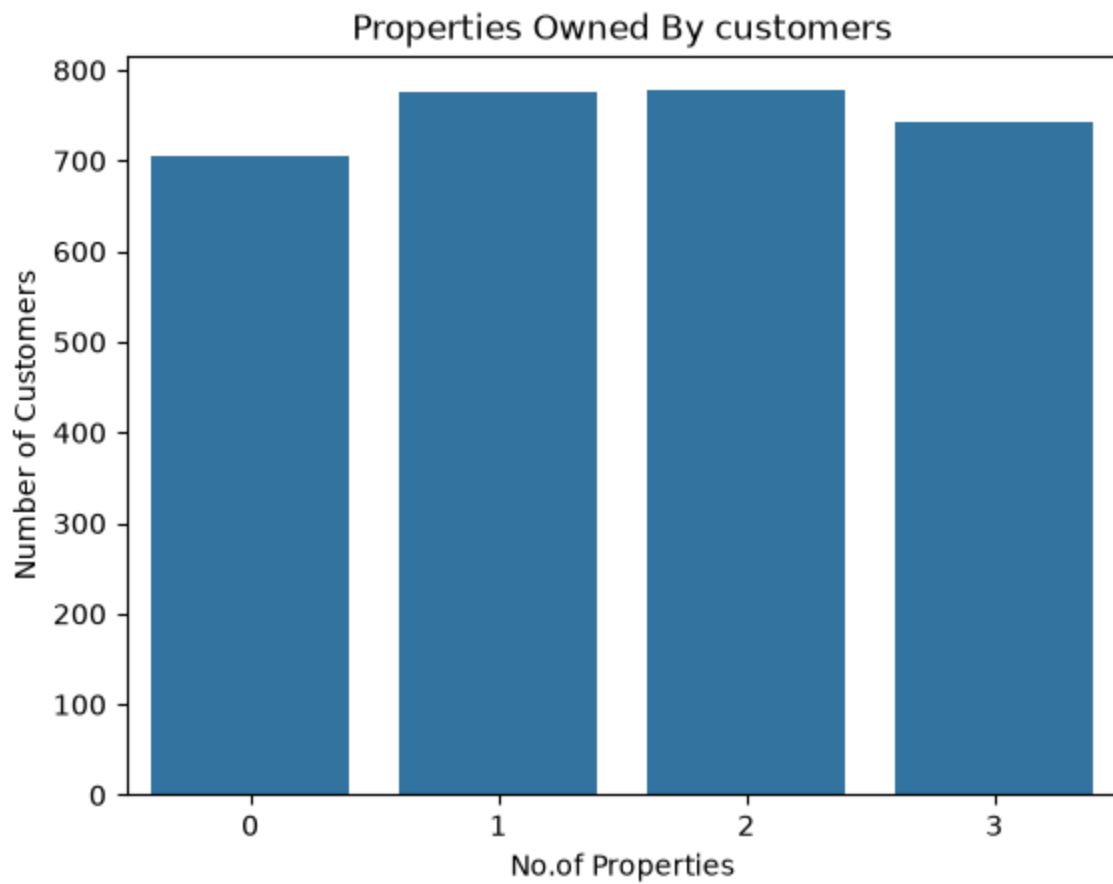
```
In [100... df["Properties Owned"].value_counts().sort_index()
```

Out[100... Properties Owned

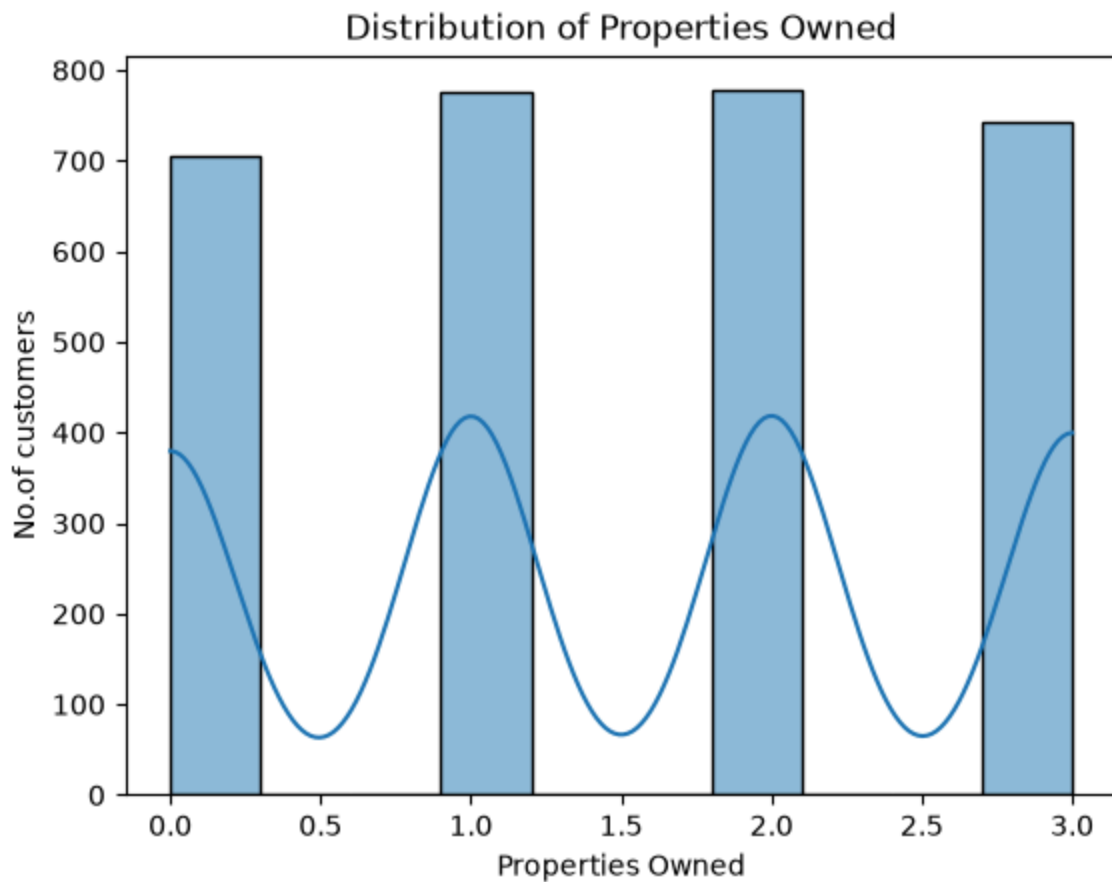
0	705
1	776
2	777
3	742

Name: count, dtype: int64

```
In [101... sns.countplot(x="Properties Owned", data=df)
plt.title("Properties Owned By customers")
plt.xlabel("No.of Properties")
plt.ylabel("Number of Customers")
plt.show()
```



```
In [109... sns.histplot(x=df["Properties Owned"], bins=10,kde=True)
plt.title("Distribution of Properties Owned")
plt.xlabel("Properties Owned")
plt.ylabel("No.of customers")
plt.show()
```



```
In [103...] df.nlargest(10, "Properties Owned")[["Client ID", "Name", "Properties Owned"]]
```

```
Out[103...]

```

	Client ID	Name	Properties Owned
10	PKR76263	Amy Martinez	3
12	PKR28766	Wayne Foster	3
15	PKR74197	Andrew Mills	3
17	PKR56539	Aaron Day	3
18	PKR53604	Kevin Weaver	3
21	PKR16101	Roger Boyd	3
25	PKR21279	Christopher Evans	3
30	PKR55475	Henry Grant	3
33	PKR53121	Timothy Stanley	3
34	PKR25680	Billy Wilson	3

```
In [105...] df["Risk Weighting"].mean()
```

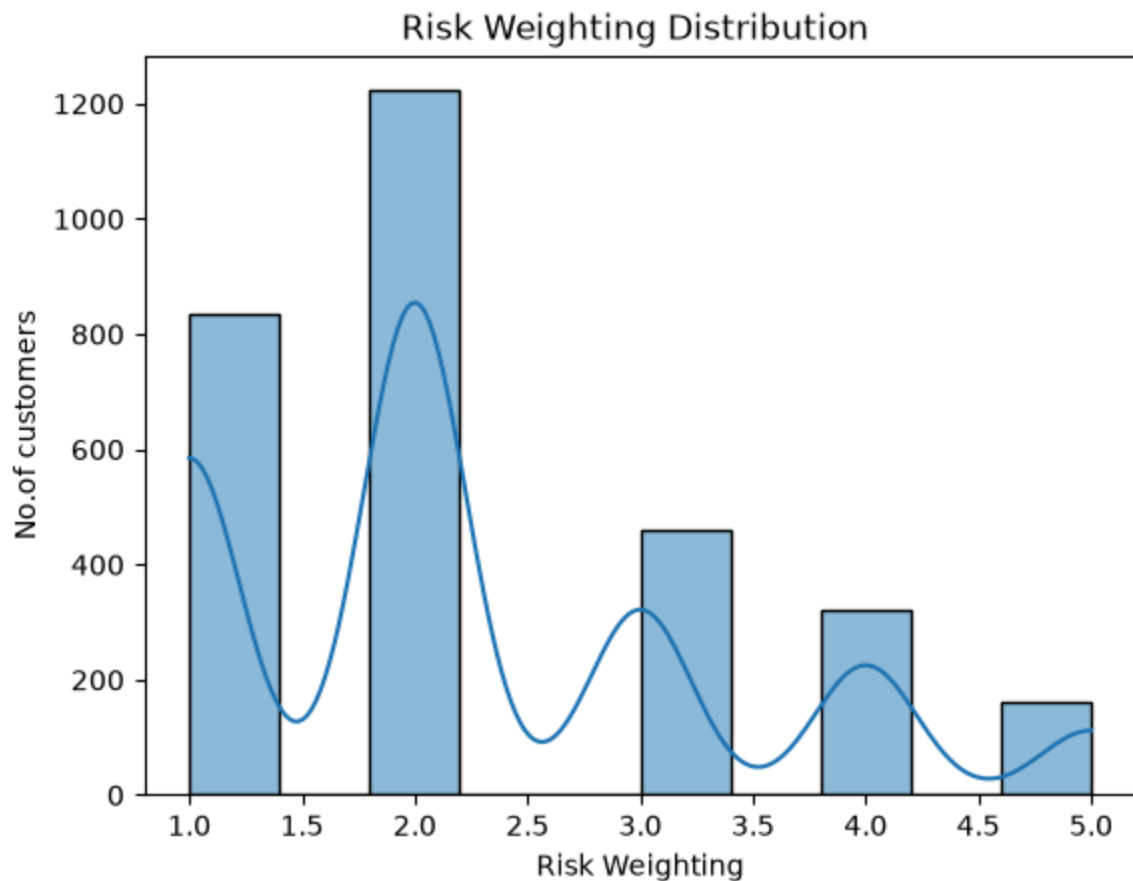
```
Out[105...] np.float64(2.2493333333333334)
```

```
In [106... print("Minimum Risk:", df["Risk Weighting"].min())
print("Maximum Risk:", df["Risk Weighting"].max())
```

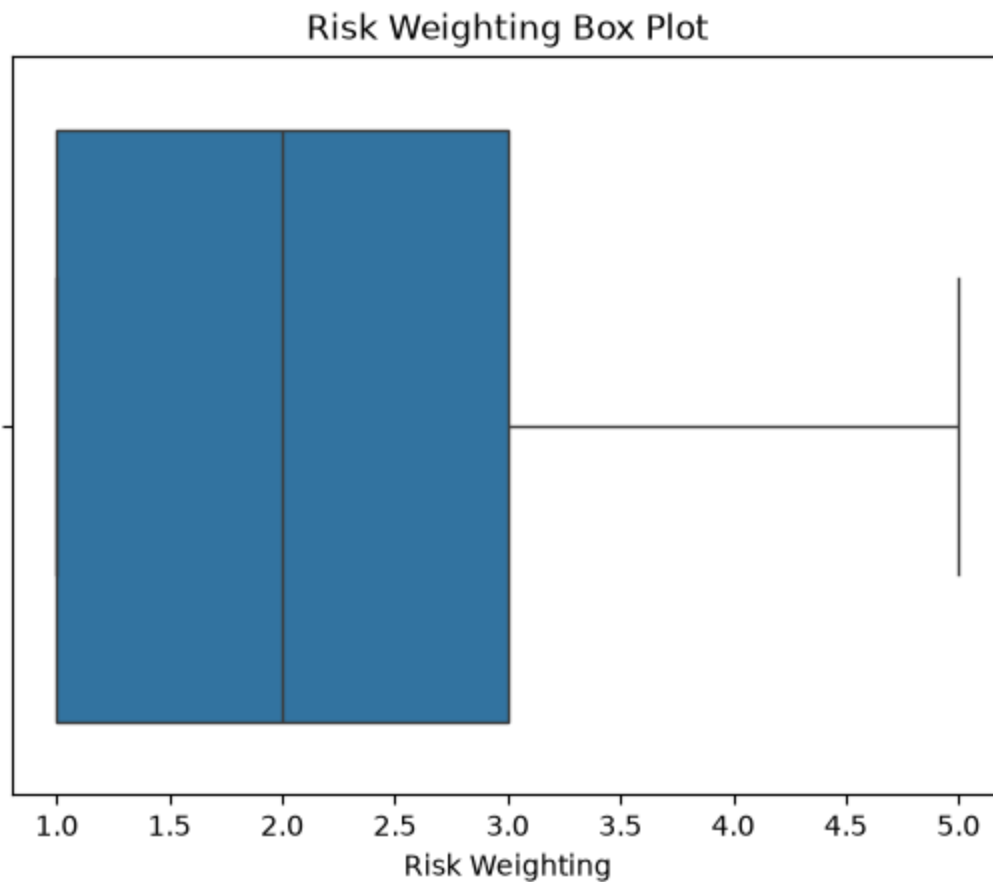
Minimum Risk: 1

Maximum Risk: 5

```
In [108... sns.histplot(x=df["Risk Weighting"], bins=10, kde=True)
plt.title("Risk Weighting Distribution ")
plt.xlabel("Risk Weighting")
plt.ylabel("No.of customers")
plt.show()
```



```
In [110... sns.boxplot(x=df["Risk Weighting"])
plt.title("Risk Weighting Box Plot")
plt.show()
```



```
In [113... df.nlargest(10, "Risk Weighting")[["Client ID", "Name", "Risk Weighting"]]
```

Out[113...

	Client ID	Name	Risk Weighting
22	PKR93121	Aaron Marshall	5
85	PKR31656	Phillip Hansen	5
96	PKR48116	Lawrence Gibson	5
99	PKR72586	Craig Wagner	5
136	PKR86466	John Hart	5
179	PKR67912	Mildred Patterson	5
208	PKR86081	Alice Gibson	5
233	PKR37844	Scott Patterson	5
272	PKR57163	Philip Hansen	5
273	PKR42982	Anthony Wagner	5

```
In [114... df.nsmallest(10, "Risk Weighting")[["Client ID", "Name", "Risk Weighting"]]
```

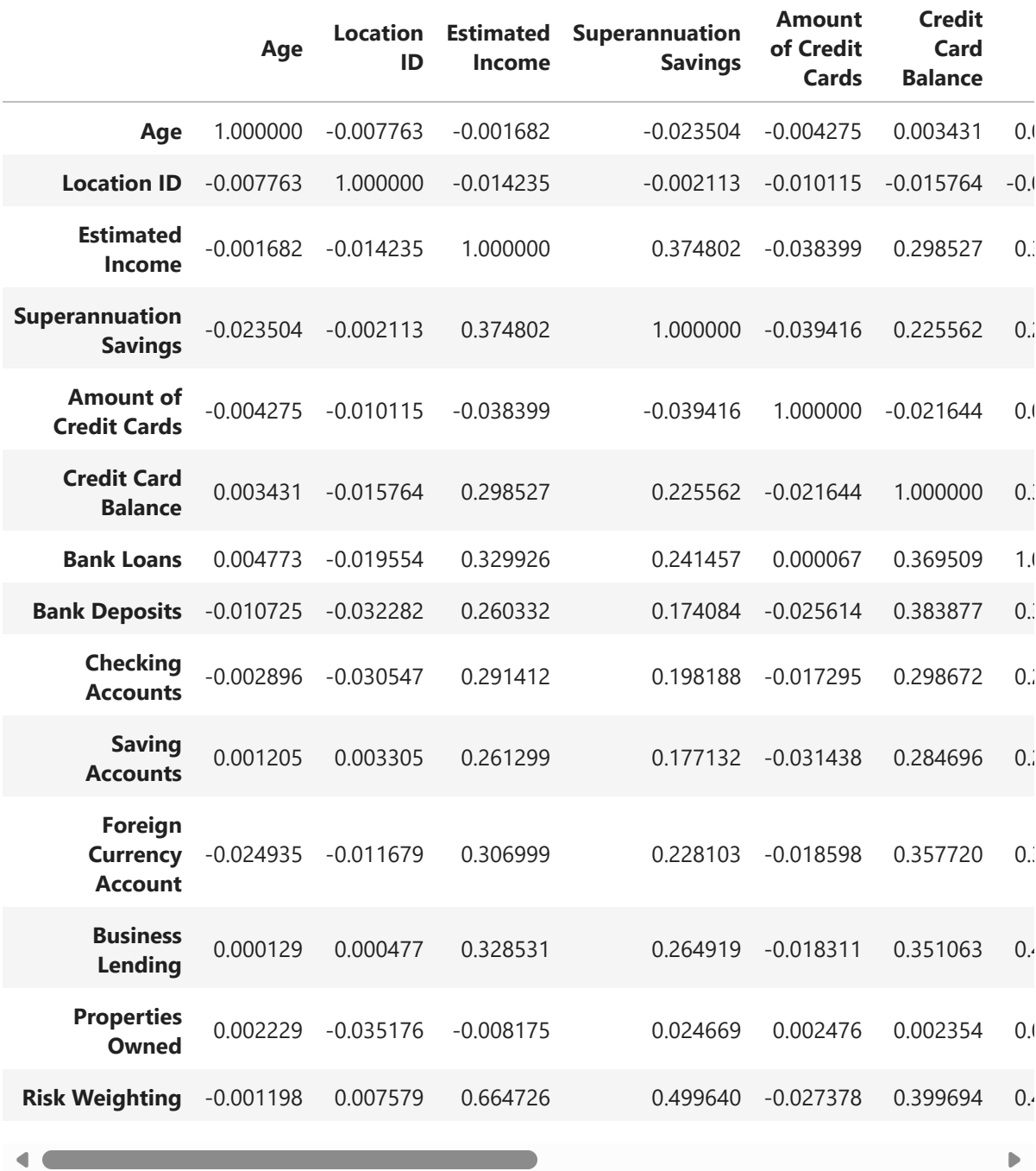
Out[114...

	Client ID	Name	Risk Weighting
6	PKR95683	Timothy Alexander	1
8	PKR13570	Philip Day	1
9	PKR53299	Jason Sims	1
10	PKR76263	Amy Martinez	1
12	PKR28766	Wayne Foster	1
17	PKR56539	Aaron Day	1
18	PKR53604	Kevin Weaver	1
20	PKR72934	Carlos Little	1
24	PKR71301	Anne Nguyen	1
25	PKR21279	Christopher Evans	1

In [115...

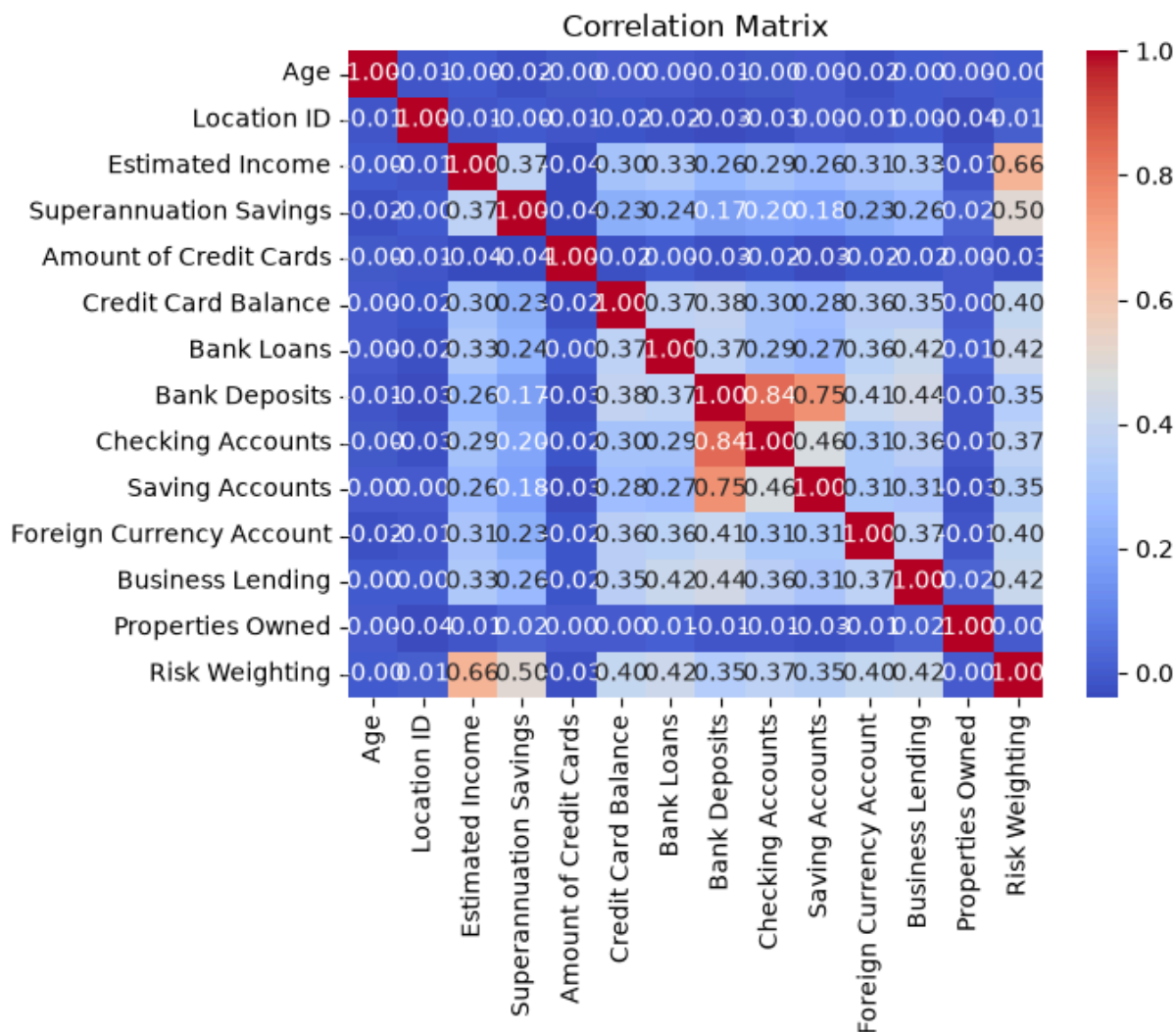
```
correlation = df.corr(numeric_only=True)
correlation
```

Out[115...



In [121...

```
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Matrix")
plt.show()
```



```
In [118... df[["Estimated Income","Bank Deposits"]].corr()
```

Out[118...

	Estimated Income	Bank Deposits
Estimated Income	1.000000	0.260332
Bank Deposits	0.260332	1.000000

```
In [119... df[["Estimated Income","Bank Loans"]].corr()
```

Out[119...

	Estimated Income	Bank Loans
Estimated Income	1.000000	0.329926
Bank Loans	0.329926	1.000000

```
In [120... df[["Estimated Income","Credit Card Balance"]].corr()
```

Out[120...

	Estimated Income	Credit Card Balance
Estimated Income	1.000000	0.298527
Credit Card Balance	0.298527	1.000000

In [122...

```
df[["Bank Deposits","Bank Loans"]].corr()
```

Out[122...

	Bank Deposits	Bank Loans
Bank Deposits	1.000000	0.373155
Bank Loans	0.373155	1.000000

In [123...

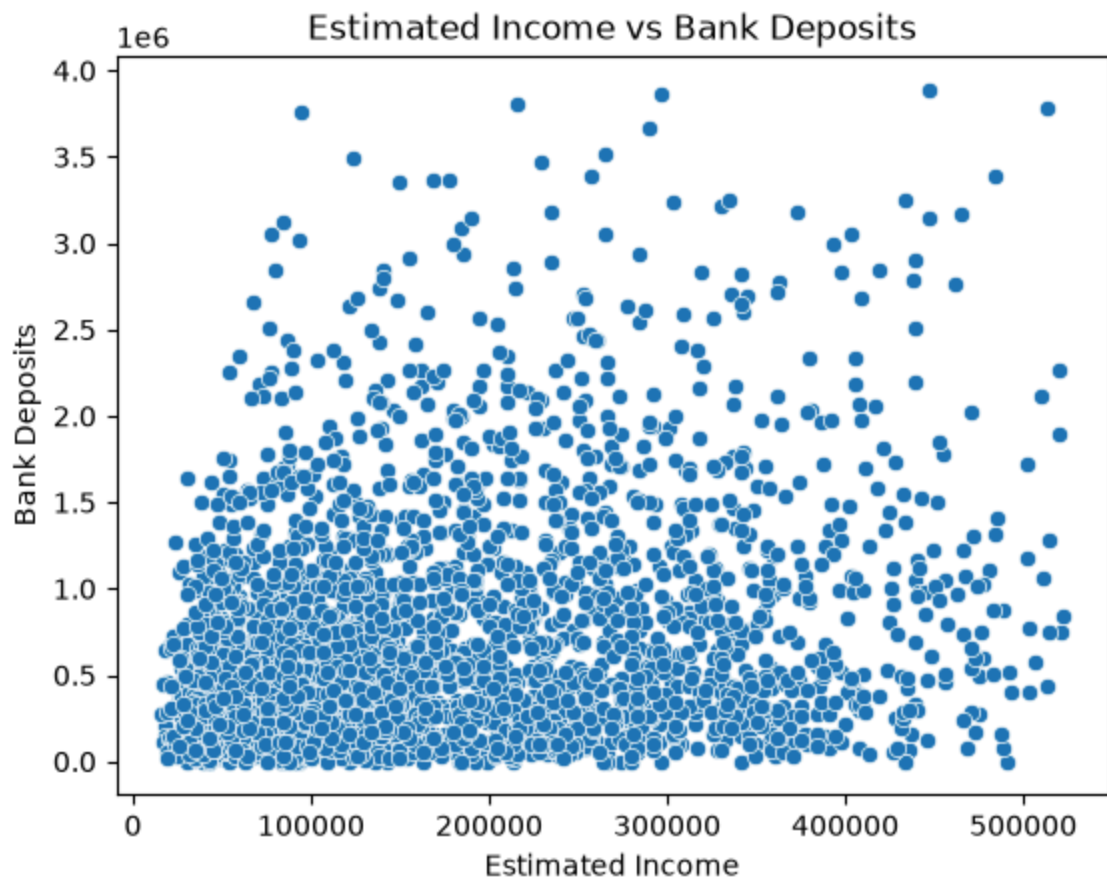
```
df[["Risk Weighting","Bank Loans"]].corr()
```

Out[123...

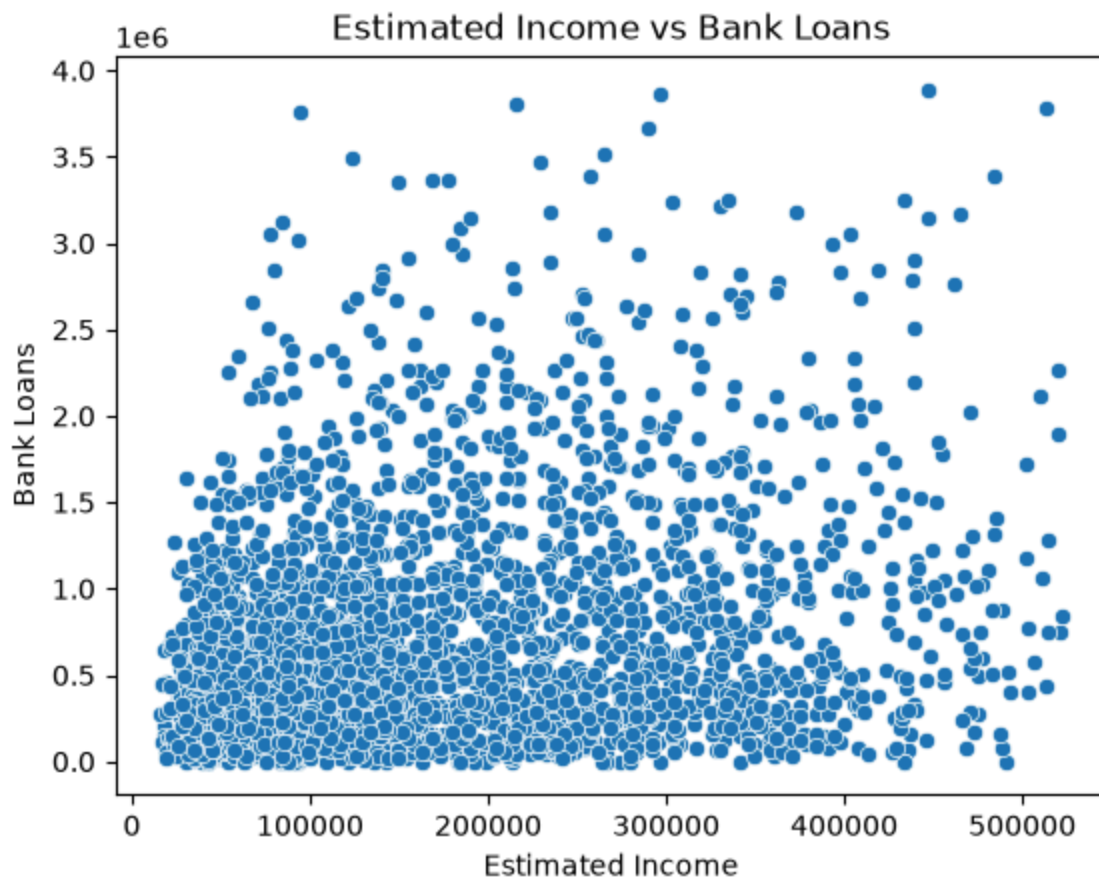
	Risk Weighting	Bank Loans
Risk Weighting	1.000000	0.421824
Bank Loans	0.421824	1.000000

In [124...

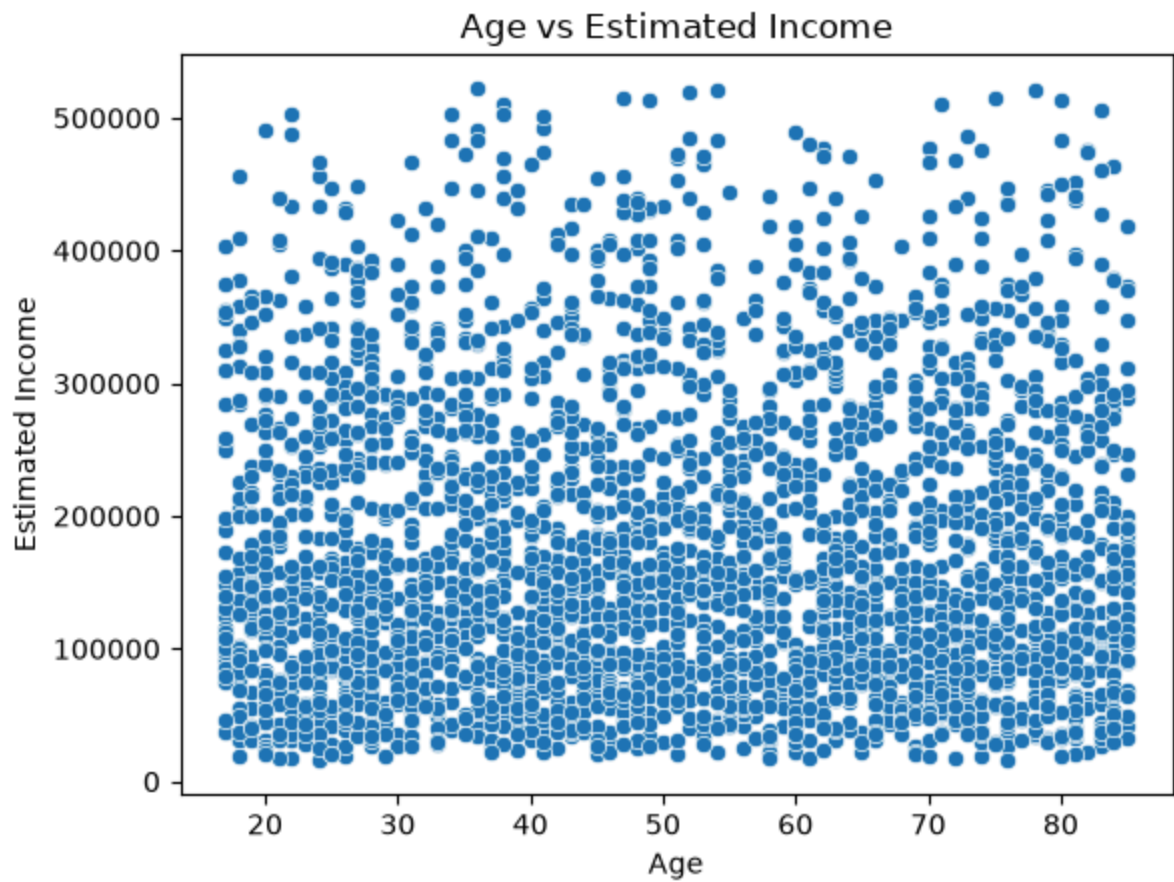
```
sns.scatterplot(x="Estimated Income", y="Bank Deposits", data=df)
plt.title("Estimated Income vs Bank Deposits")
plt.xlabel("Estimated Income")
plt.ylabel("Bank Deposits")
plt.show()
```



```
In [126... sns.scatterplot(x="Estimated Income", y="Bank Deposits", data=df)
plt.title("Estimated Income vs Bank Loans")
plt.xlabel("Estimated Income")
plt.ylabel("Bank Loans")
plt.show()
```

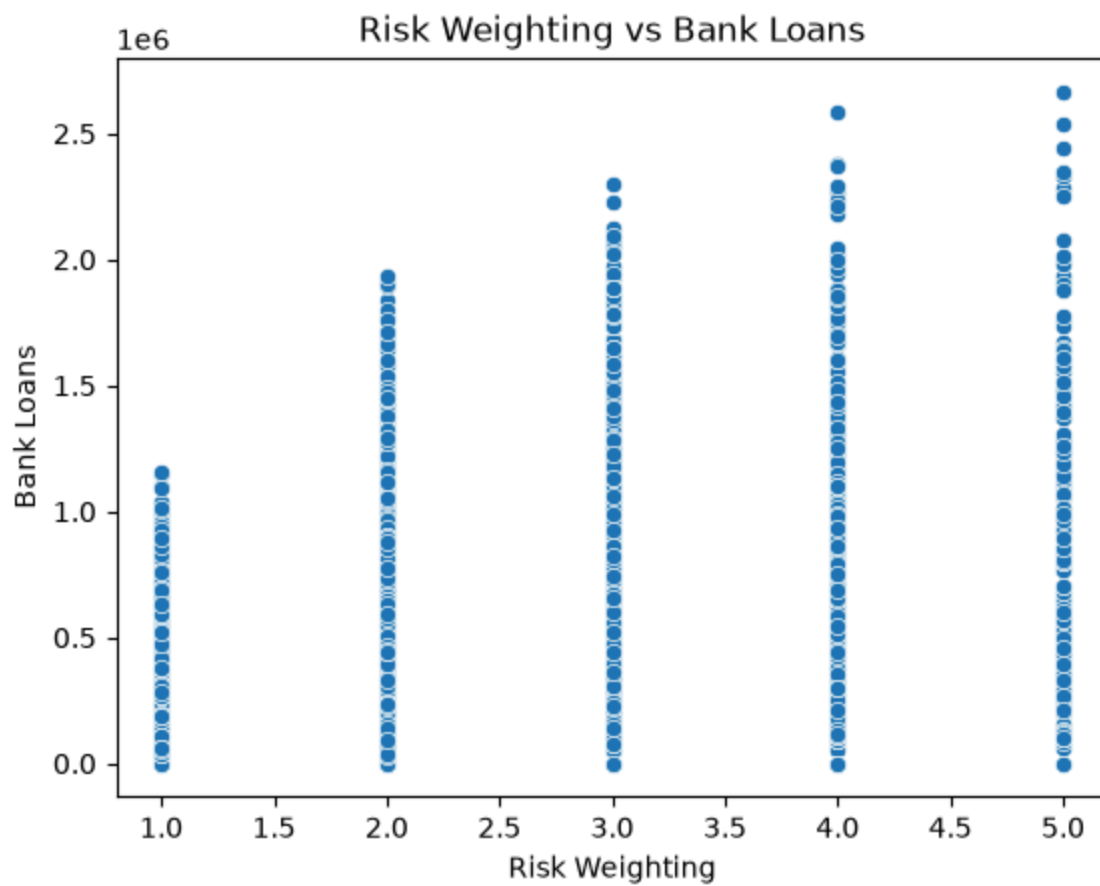


```
In [128... sns.scatterplot(x="Age", y="Estimated Income", data=df)
plt.title("Age vs Estimated Income")
plt.xlabel("Age")
plt.ylabel("Estimated Income")
plt.show()
```

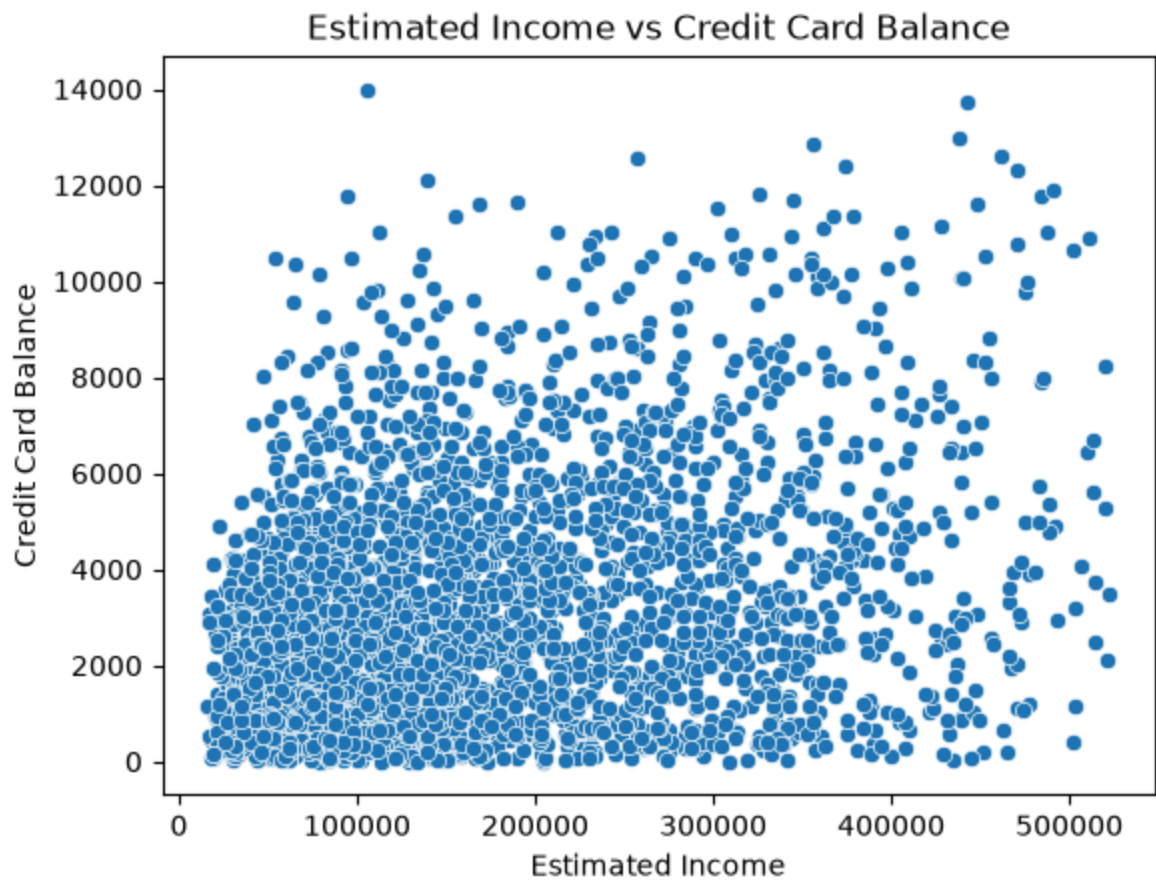


In [129...

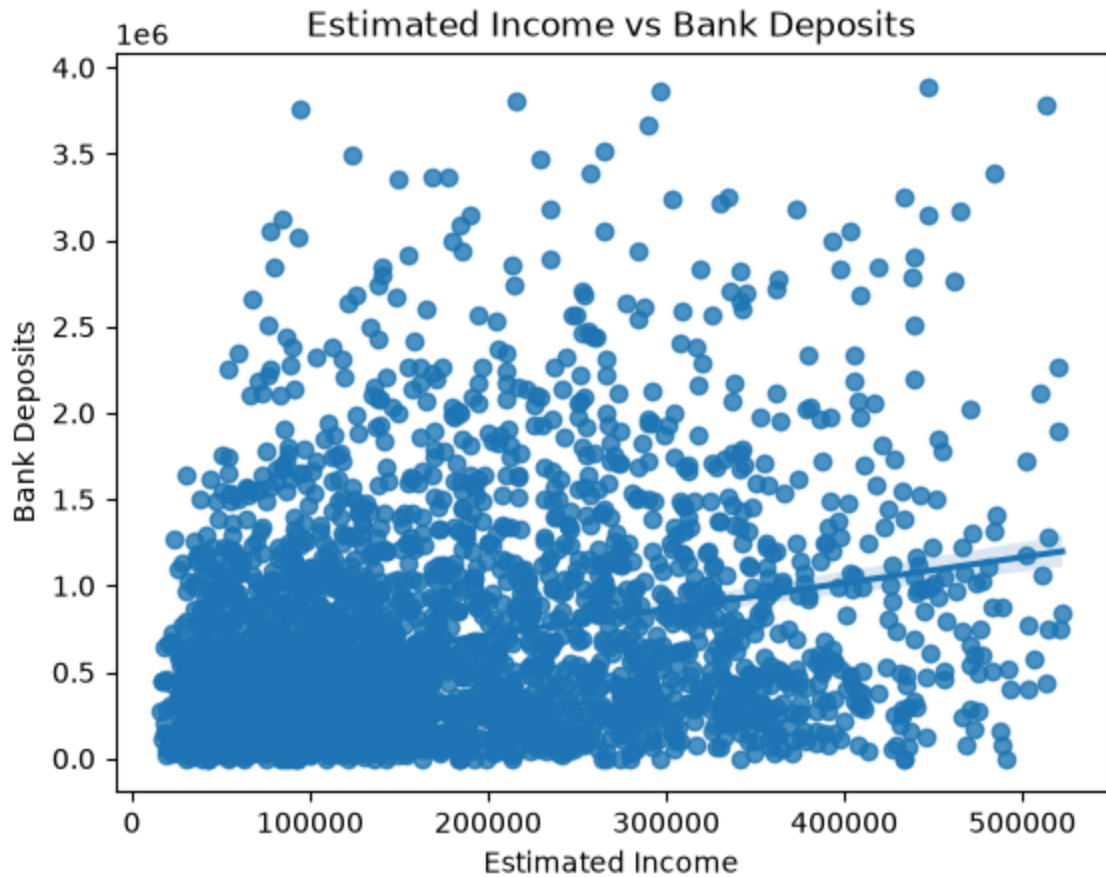
```
sns.scatterplot(x="Risk Weighting", y="Bank Loans", data=df)
plt.title("Risk Weighting vs Bank Loans")
plt.xlabel("Risk Weighting")
plt.ylabel("Bank Loans")
plt.show()
```



```
In [131... sns.scatterplot(x="Estimated Income", y="Credit Card Balance", data=df)
plt.title("Estimated Income vs Credit Card Balance")
plt.xlabel("Estimated Income")
plt.ylabel("Credit Card Balance")
plt.show()
```



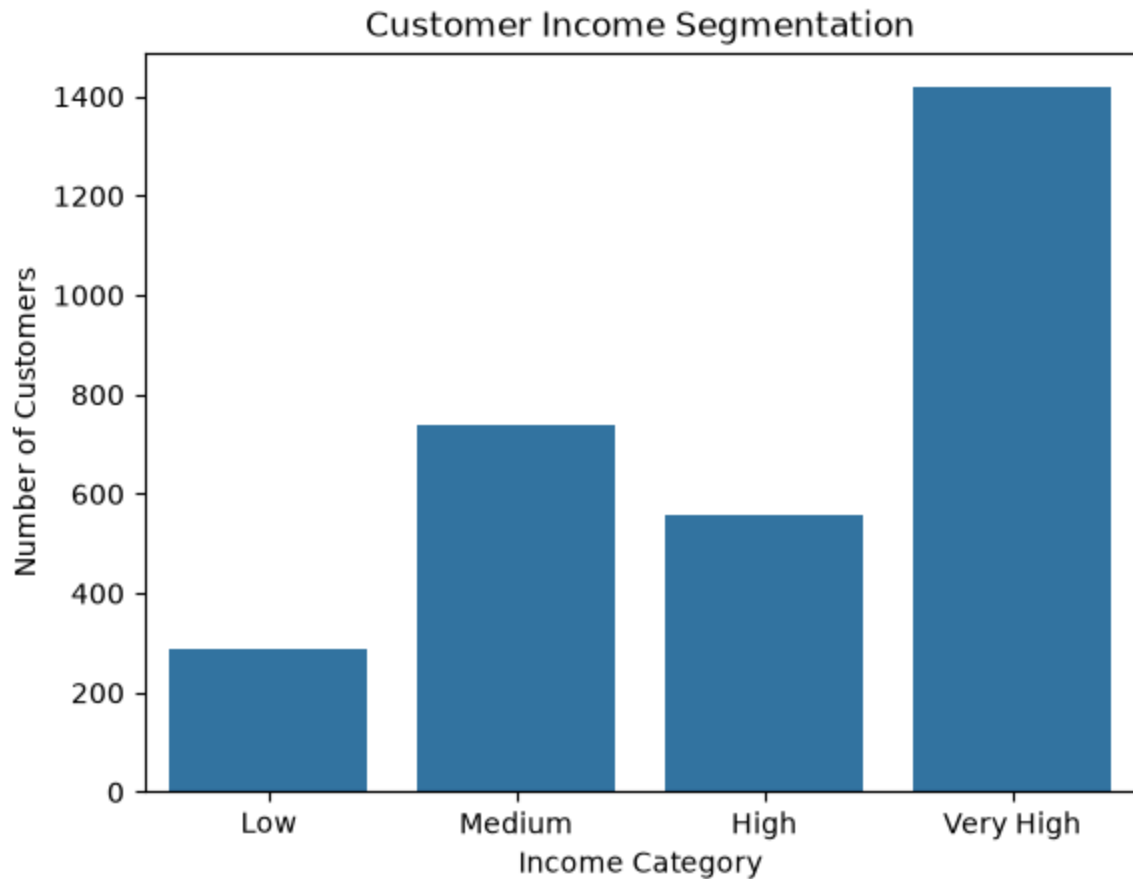
```
In [132... sns.regplot(x="Estimated Income", y="Bank Deposits", data=df)
plt.title("Estimated Income vs Bank Deposits")
plt.xlabel("Estimated Income")
plt.ylabel("Bank Deposits")
plt.show()
```



```
In [133...] income_segments = pd.cut(df["Estimated Income"], bins=[0,50000,100000,150000,float(
income_segments.value_counts())
```

```
Out[133...] Estimated Income
Very High    1417
Medium       740
High         556
Low          287
Name: count, dtype: int64
```

```
In [134...] sns.countplot(x=income_segments)
plt.title("Customer Income Segmentation")
plt.xlabel("Income Category")
plt.ylabel("Number of Customers")
plt.show()
```



```
In [135...] deposit_segments = pd.cut(df["Bank Deposits"], bins=4, labels=["Low Deposit", "Med  
deposit_segments.value_counts()
```

```
Out[135...] Bank Deposits  
Low Deposit      2288  
Medium Deposit   541  
High Deposit     137  
Very High Deposit  34  
Name: count, dtype: int64
```

```
In [136...] deposit_segments = pd.cut(df["Bank Deposits"], bins=4, labels=["Low Deposit", "Med  
deposit_segments.value_counts()
```

```
Out[136...] Bank Deposits  
Low Deposit      2288  
Medium Deposit   541  
High Deposit     137  
Very High Deposit  34  
Name: count, dtype: int64
```

```
In [137...] risk_segments = pd.cut(df["Risk Weighting"], bins=4, labels=["Low Risk", "Moderate  
risk_segments.value_counts()
```

Out[137... Risk Weighting
Low Risk 2058
Moderate Risk 460
High Risk 322
Very High Risk 160
Name: count, dtype: int64

```
In [138... df["Banking Relationship"].value_counts()
```

Out[138... Banking Relationship
Retail 1436
Institutional 729
Commercial 430
Private Bank 405
Name: count, dtype: int64

```
In [139... pd.crosstab(income_segments , df["Banking Relationship"])
```

Out[139... **Banking Relationship Commercial Institutional Private Bank Retail**

Estimated Income					
Low	38	69	51	129	
Medium	110	189	90	351	
High	87	115	67	287	
Very High	195	356	197	669	